UNIDAD 10. PERSISTENICA AVANZADA: BASES DE DATOS ORIENTADAS A OBJETOS

Índice

1. Bases de Datos Orientadas a Objetos (BDOO) 3
   1.1. Concepto y evolución: De las RDBMS a las BDOO 3
   1.2. El estándar ODMG: El modelo de objetos (OM) y el lenguaje de definición (ODL) 5
   1.3. Eliminación de la "Desadaptación de Impedancia": Por qué las BDOO son más naturales 7
2. Características de las BDOO 9
   2.1. Identidad de objetos (OID): El identificador único independiente del valor 9
   2.2. Jerarquías y relaciones nativas 10
3. Instalación del gestor de BDOO 12
   3.1. Elección del motor: Gestores populares (db4o, ObjectDB, Perst) 12
   3.2. Configuración del entorno: Importación de librerías JAR 14
   3.3. Modos de funcionamiento: Servidor vs. Embebida 16
4. Creación de bases de datos 18
   4.1. El contenedor de objetos: Apertura y cierre de sesiones (`ObjectContainer`) 18
   4.2. Estructura física: Ficheros de datos y gestión del almacenamiento 20
   4.3. Configuración inicial: Índices, cascada y profundidad 23
5. Mecanismos de consulta 25
   5.1. Consultas por Ejemplo (QBE - Query by Example) 25
   5.2. Consultas Nativas (Native Queries) 27
   5.3. SODA API (Simple Object Database Access) 29
6. Lenguaje de consultas: Sintaxis, expresiones y operadores 32
   6.1. OQL (Object Query Language): El estándar SQL-like 32
   6.3. Operadores lógicos y de comparación: Gestión de predicados complejos 34
   6.4. Navegación por punteros: Uso del operador punto (.) 36
7. Recuperación, modificación y borrado de información 39
   7.1. Almacenamiento y persistencia: El método `store()` / `persist()` 39
   7.2. Recuperación y refresco de objetos 42
   7.3. Eliminación de información: Borrado de instancias y gestión de referencias huérfanas 43
   7.4. Gestión de Transacciones: Garantizando ACID con commit() y rollback() 45
8. Tipos de datos objeto: atributos y métodos 48

8.1. Persistencia de atributos: Tipos simples, objetos anidados y tipos transient 48
8.2. Almacenamiento de comportamiento: ¿Cómo gestiona la base de datos los métodos? 50
8.3. Profundidad de activación (Activation Depth) 52
9. Tipos de datos colección 55
9.1. Persistencia de Arrays y Listas: Manejo de ArrayList, LinkedList y arrays nativos 55
9.2. Conjuntos y Mapas: Uso de HashSet y HashMap 56
9.3. Gestión de relaciones: Implementación de cardinalidades 1:N y N:M 58
Resumen 60
1. El Concepto Fundamental 60
2. Características Clave (Vs. SQL) 61
3. Motores y Arquitectura 61
4. Operaciones CRUD (Ciclo de Vida) 61
5. Mecanismos de Consulta 61
6. Configuración Crítica 62
7. Colecciones y Relaciones 62

1. Bases de Datos Orientadas a Objetos (BDOO)

1.1. Concepto y evolución: De las RDBMS a las BDOO

Una **Base de Datos Orientada a Objetos (BDOO)** es un sistema de gestión de bases de datos en el que la información se representa mediante **objetos**, exactamente igual que en la Programación Orientada a Objetos (POO).

En una BDOO, no existen tablas, ni columnas, ni filas, ni claves foráneas. Existen Clases, Atributos, Métodos y referencias directas entre objetos.

1. La Evolución Histórica

Para entender por qué surgieron, hay que ver el problema que intentaban resolver.

A. La Era Relacional (RDBMS) - Años 70-90. Las bases de datos relacionales (como Oracle, SQL Server, MySQL) dominaron el mundo.

- **Filosofía:** Todo son datos planos (filas y columnas).
- **Ventaja:** Matemáticamente perfectas, muy eficientes para datos simples (contabilidad, inventarios) y búsquedas masivas.
- **Lenguaje:** SQL.

B. El Auge de la POO - Años 90. Lenguajes como C++ y Java se volvieron el estándar. Los programadores empezaron a modelar sistemas complejos con herencia, colecciones y grafos de objetos anidados.

C. El Problema: La Desadaptación de Impedancia (Impedance Mismatch). Aquí chocaron los dos mundos. Imagina que tienes un objeto Coche que tiene un Motor, y el motor tiene Piezas.

- **En Java:** Es un solo objeto `Coche` con referencias dentro.
- **En SQL:** Tienes que guardar el coche en la tabla `Coches`, el motor en la tabla `Motores` y las piezas en la tabla `Piezas`. Luego tienes que crear IDs artificiales para unirlos.

  La Analogía del Garaje:

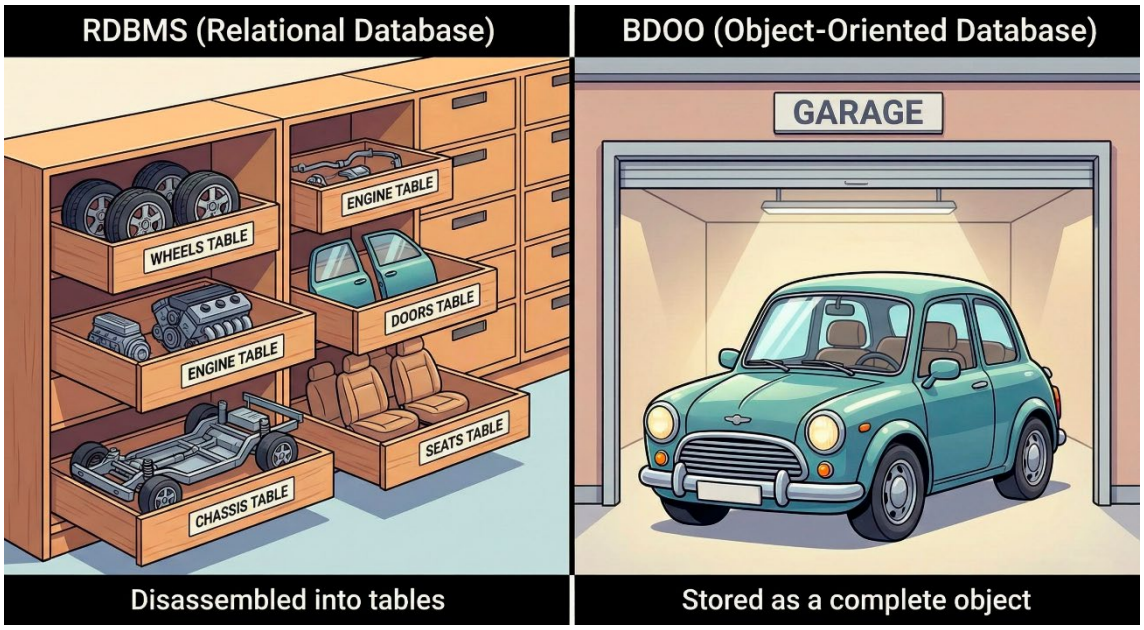
  - **RDBMS (Relacional):** Para guardar tu coche en el garaje, primero tienes que desmontarlo pieza por pieza (ruedas, motor, puertas) y guardar cada tipo de pieza en un cajón diferente. Cuando quieras usar el coche, tienes que sacarlas y volver a montarlo.
  - **BDOO (Objetos):** Aparcas el coche entero en el garaje. Cuando lo necesitas, lo sacas entero y arrancas.

2. La Solución BDOO

Las Bases de Datos Orientadas a Objetos prometían ser el "Santo Grial": persistencia transparente.

- **Concepto:** El programador Java simplemente dice `baseDatos.store(miCoche)` y el sistema guarda el grafo de objetos completo, manteniendo su estructura, tipos y relaciones.

- **Sin SQL:** No hace falta aprender otro lenguaje. Se consulta usando el propio lenguaje de programación (Java).



3. RDBMS vs. BDOO

Característica	Base de Datos Relacional (SQL)	Base de Datos Orientada a Objetos
Unidad de datos	La Fila (Tupla).	El Objeto.
Estructura	Tablas planas y relaciones por claves (FK).	Grafos de objetos y referencias directas.
Esquema	Rígido (tienes que hacer `ALTER TABLE` si cambia algo).	Flexible (se adapta a la clase Java).
Identificación	Clave Primaria (Primary Key) definida por el usuario.	OID (Object ID) interno y transparente.
Rendimiento	Lento en relaciones complejas (muchos `JOINs`).	Rapidísimo en navegación de objetos complejos.

¿Por qué no usamos todos BDOO hoy en día?

Aunque las BDOO son técnicamente superiores para Java, las Relacionales ganaron la guerra comercial porque el SQL ya estaba en todas partes. Como solución intermedia, la industria adoptó los **ORM (Object-Relational Mappers)** como *Hibernate*, que automatizan el trabajo sucio de "desmontar el coche", permitiendo usar bases SQL como si fueran de objetos (aunque con menor rendimiento).

Sin embargo, las BDOO puras (como **ObjectDB** o **db4o**) siguen vivas y son imbatibles en sistemas de tiempo real, sistemas embebidos y aplicaciones con estructuras de datos muy complejas.

1.2. El estándar ODMG: El modelo de objetos (OM) y el lenguaje de definición (ODL)

A principios de los 90, cuando las bases de datos orientadas a objetos empezaron a proliferar, cada empresa (ObjectStore, Versant, O2) tenía su propia forma de guardar y consultar objetos. Si escribías código para una, no servía para la otra.

Para solucionar esto, en 1991 se formó el **ODMG (Object Data Management Group)**. Su misión fue crear un estándar abierto para que el almacenamiento de objetos fuera portable, igual que SQL lo es para las relacionales.

Aunque el grupo se disolvió en 2001, su trabajo sentó las bases de lo que hoy usamos en Java (JPA/JDO). El estándar se compone de varias partes, pero las fundamentales para definir la estructura son el OM y el ODL.

1. El Modelo de Objetos (OM - Object Model)

El **Modelo de Objetos** es la teoría detrás del estándar. Define "qué es una base de datos de objetos" y cómo debe comportarse. Se basó fuertemente en el modelo de objetos de la OMG (usado en CORBA), añadiendo características de persistencia.

El OM establece que la unidad básica de almacenamiento es el **Objeto**, y define dos conceptos clave que la base de datos debe distinguir:

- **El Objeto (Object):** Tiene un identificador único (**OID**) y es mutable (su contenido puede cambiar, pero sigue siendo el mismo objeto). Ejemplo: Un objeto `Cliente`.
- **El Literal (Literal):** Es un valor que no tiene identificador propio, se identifica por su valor y es inmutable. Ejemplo: El número `25` o la cadena `"Madrid"`.

Además, el OM define las reglas del juego que ya conoces de Java, pero aplicadas al disco duro:

- **Tipos y Clases:** La base de datos debe entender la diferencia entre Interfaz (qué hace) y Clase (cómo lo hace).
- **Herencia:** Debe soportar jerarquías (Si busco "Vehículos", debe devolverme también los objetos "Coche" guardados).
- **Encapsulación:** Los atributos solo deberían ser accesibles mediante métodos (aunque en la práctica de BDOO esto se relaja por eficiencia).

2. El Lenguaje de Definición de Objetos (ODL - Object Definition Language)

Si en SQL usamos `CREATE TABLE` (DDL) para definir el esquema, en el mundo de objetos usamos **ODL**.

El ODL es un lenguaje de especificación (muy parecido a Java o C++) que sirve para describir la estructura de la base de datos de forma independiente al lenguaje de programación que uses. Es el "plano de arquitectura" de tus datos.

Características del ODL

- No es un lenguaje de programación completo (no tiene bucles ni `if`).
- Solo define **interfaces** y **atributos**.
- Permite definir relaciones bidireccionales.

Ejemplo de sintaxis ODL

Fíjate cómo se parece a una interfaz de Java, pero define explícitamente las relaciones inversas:

Fragmento de código

```
// Definición teórica en ODL
class Departamento {
    attribute string nombre;
    attribute long presupuesto;

    // Relación 1:N (Un departamento tiene muchos empleados)
    relationship Set<Empleado> tiene_empleados
        inverse Empleado::trabaja_en;
};

class Empleado {
    attribute string nombre;
    attribute date fecha_alta;

    // Relación N:1 (Un empleado trabaja en un departamento)
    relationship Departamento trabaja_en
        inverse Departamento::tiene_empleados;
};
```

3. La realidad práctica: El "Binding" Java

Te preguntarás: *"¿Tengo que aprender este lenguaje ODL y escribir ficheros de texto aparte?"*

La respuesta hoy en día es NO.

El estándar ODMG definió el concepto de **Language Binding (Vinculación de Lenguaje)**. Esto significa que existe una traducción directa entre ODL y Java (o C++, o Smalltalk).

En una BDOO moderna para Java (como ObjectDB o db4o), **tu código Java ES el ODL**.

- No escribes un `CREATE TABLE`.
- No escribes un archivo `.odl`.
- Simplemente escribes tu `class Empleado` en Java. La base de datos analiza tu clase (`.class`) y crea el esquema internamente basándose en ella.

  Resumen: El ODMG definió las reglas teóricas. El ODL es la forma de describir el esquema en papel. Pero en la práctica, al programar en Java, el propio lenguaje Java actúa como ODL, haciendo que la definición de la base de datos sea totalmente transparente para ti.

1.3. Eliminación de la "Desadaptación de Impedancia": Por qué las BDOO son más naturales

El término **"Impedance Mismatch"** (Desadaptación de Impedancia) proviene de la ingeniería eléctrica. Ocurre cuando intentas conectar dos circuitos que tienen resistencias diferentes: la energía se pierde en la conexión y el sistema no fluye.

En el desarrollo de software, usamos este término para describir el **conflicto fundamental** entre dos paradigmas que son estructuralmente incompatibles:

1. **El Paradigma Orientado a Objetos (Java):** Se basa en grafos de objetos, herencia, polimorfismo y referencias en memoria.
2. **El Paradigma Relacional (SQL):** Se basa en tablas planas, álgebra relacional y claves primarias/foráneas.

1. Los puntos de fricción (El "Dolor" del SQL)

Cuando intentas guardar un objeto Java complejo en una base de datos relacional (MySQL, Oracle), te encuentras con problemas estructurales que te obligan a escribir mucho "código fontanero" (o usar frameworks pesados como Hibernate) para traducir un mundo al otro.

Aquí están los principales choques:

A. El problema de la Granularidad

- **Java:** Tienes una clase `Persona` que tiene dentro un objeto `Direccion` (calle, ciudad, CP). Para Java, `Direccion` es solo una parte pequeña de `Persona`.
- **SQL:** No puedes guardar un objeto dentro de una celda de una tabla. Tienes dos opciones feas: o aplanas los datos (columnas `calle`, `ciudad` en la tabla Persona) o creas una tabla `Direcciones` separada.

B. El problema de la Herencia

- **Java:** `Gerente` extiende de `Empleado`. Un `Gerente` **ES** un `Empleado`.
- **SQL:** Las tablas no heredan. ¿Cómo guardas esto?
    - ¿Una tabla gigante con columnas vacías? (Ineficiente).
    - ¿Dos tablas separadas y duplicas datos? (Malo para la integridad).
    - ¿Una tabla padre y una tabla hijo unidas por ID? (Lento por los JOINs).

C. El problema de la Identidad

- **Java:** Dos objetos son iguales si apuntan a la misma dirección de memoria (`a == b`), independientemente de sus datos.
- **SQL:** Dos filas son iguales si tienen la misma Clave Primaria (PK). Tienes que "contaminar" tus clases Java añadiendo campos `id` que no tienen nada que ver con la lógica de negocio, solo para satisfacer a la base de datos.

D. El problema de las Asociaciones (El más grave)

- **Java:** Las relaciones son referencias directas. `coche.getMotor().getPiezas()`. Navegar por ahí es instantáneo (nanosegundos) porque está en memoria.
- **SQL:** Las relaciones son claves foráneas. Para reconstruir el coche, la base de datos tiene que buscar en índices, hacer `JOIN` de tres tablas y devolver los datos. Es miles de veces más lento.

2. La Solución BDOO: Eliminación de la Barrera

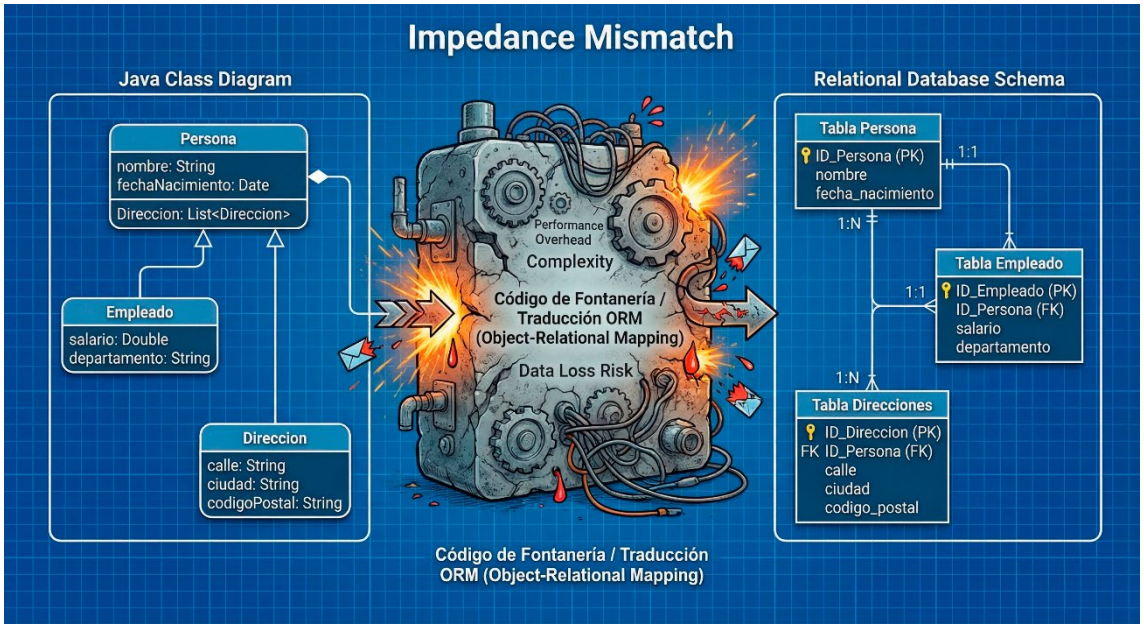
Las Bases de Datos Orientadas a Objetos eliminan esta fricción por una razón simple: **hablan el mismo idioma que Java.**

Al usar una BDOO (como ObjectDB o db4o), la base de datos adopta el modelo de Java, no al revés.

- **No hay traducción:** No hay que "desmontar" el objeto en filas. El objeto se serializa y se guarda tal cual.
- **Persistencia Transparente:**
  - ¿Tienes Herencia? La BDOO la entiende y guarda la jerarquía.
  - ¿Tienes una lista `ArrayList<String>` dentro de tu objeto? La BDOO guarda la lista.
  - ¿Tienes un grafo cíclico (A apunta a B y B apunta a A)? La BDOO mantiene las referencias.

3. ¿Por qué es más natural para el desarrollador?

Para ti, como programador Java, la diferencia es abismal en términos de productividad:



2. Características de las BDOO

2.1. Identidad de objetos (OID): El identificador único independiente del valor

En el mundo de las bases de datos, saber distinguir una entidad de otra es fundamental. Sin embargo, la forma en que lo hacen las bases de datos Relacionales (SQL) y las Orientadas a Objetos (BDOO) es radicalmente distinta.

Las BDOO introducen el concepto de **OID (Object Identifier)**.

1. ¿Qué es un OID?

El OID es un identificador único, inmutable y generado por el sistema, que se asigna a cada objeto en el momento de su creación y persiste durante toda su vida útil.

La característica clave es que **es independiente del valor del objeto**.

- No depende de los datos que contiene el objeto (como el DNI, el email o el nombre).
- Es invisible para el usuario (generalmente).
- Funciona como un "puntero persistente".

  Analogía:

  - **SQL (Clave Primaria):** Es como tu **DNI** o tu número de Seguridad Social. Es un número que está escrito en tu tarjeta. Si el gobierno decide cambiar el sistema de numeración, tienes un problema administrativo enorme.
  - **BDOO (OID):** Es como tu **ADN**. Es intrínseco a ti. Aunque te cambies el nombre, te mudes de país o pierdas el DNI, sigues siendo tú. No necesitas llevar un número tatuado para ser único.

2. Identidad por Valor vs. Identidad por Objeto

Para entender la potencia del OID, hay que compararlo con lo que hacíamos en SQL.

A. En Bases de Datos Relacionales (Identidad por Valor). La identidad se basa en el contenido de una columna (Clave Primaria o PK).

- **Problema 1:** Si usas una clave natural (ej: el Email) y el usuario cambia de email, tienes que actualizar esa tabla y **todas** las tablas que tengan ese email como clave foránea (efecto cascada).
- **Problema 2:** Si usas una clave artificial (`ID_CLIENTE` autoincremental), estás "ensuciando" tu modelo de negocio con datos técnicos que no aportan valor real al objeto, solo sirven para la base de datos.

B. En BDOO (Identidad por Objeto). La identidad es gestionada internamente por el motor de base de datos.

- El objeto `Cliente` en Java no necesita tener un atributo `int id`.

- La base de datos le asigna un número interno (ej: `#30492`) que tú nunca tocas.
- Si cambias el nombre, el email o cualquier dato del cliente, el OID sigue siendo `#30492`.
- Las relaciones con otros objetos apuntan a ese OID. Si el objeto cambia de valor, los enlaces no se rompen.

3. Identidad vs. Igualdad

En Java y en las BDOO, es crucial distinguir entre dos objetos que son "el mismo" y dos objetos que "parecen el mismo".

1. **Identidad (Identity):** ¿Son el mismo objeto físico?

   - En Java: `a == b` (Compara referencias de memoria).
   - En BDOO: Comparación de OIDs. Dos referencias apuntan al mismo registro único en disco.

2. **Igualdad (Equality):** ¿Tienen el mismo contenido?

   - En Java: `a.equals(b)` (Compara los valores internos).
   - Dos objetos pueden tener exactamente los mismos datos (Nombre: "Juan", Edad: 25) y tener OIDs diferentes (son dos gemelos distintos).

4. Ventajas del OID para el Programador

1. **Código más limpio:** Tus clases Java (`Persona`, `Factura`) solo tienen los atributos de negocio. No hace falta llenarlas de campos `id`, `fk_cliente`, etc.
2. **Rendimiento:** La base de datos usa el OID para localizar el objeto en el disco directamente, sin tener que buscar en un índice B-Tree como hacen las relacionales con las Primary Keys.
3. **Integridad de Referencias:** Como el OID nunca cambia, es imposible que una referencia se quede "colgando" o apunte al sitio incorrecto porque alguien modificó una clave primaria.

2.2. Jerarquías y relaciones nativas

En Java, diseñar un sistema con herencia y polimorfismo es el pan de cada día. Sin embargo, en SQL, conceptos como `extends` o `implements` simplemente **no existen**.

Las BDOO eliminan la necesidad de "trucos" para guardar estas estructuras.

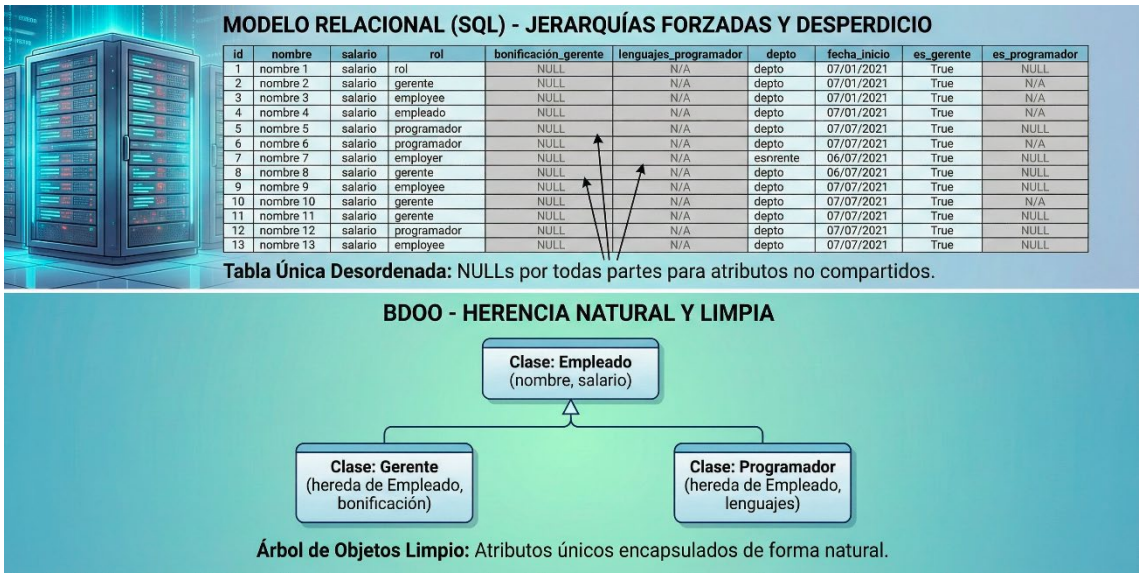
1. Soporte directo para la Herencia

Imagina una jerarquía clásica: Tienes una clase padre `Empleado`, y dos hijas: `Programador` (tiene un atributo extra `lenguajes`) y `Gerente` (tiene un atributo extra `bonus`).

El Dolor en SQL (Relacional)

Para guardar esto en tablas, tienes que elegir entre tres estrategias dolorosas (que seguro has sufrido con Hibernate):

1. **Una tabla gigante (`Single Table`):** Una sola tabla con columnas para todo y muchos `NULLs`.
2. **Una tabla por clase (`Table per Class`):** Datos duplicados.
3. **Tablas unidas (`Joined`):** Una tabla padre y tablas hijas unidas por ID. Para leer un gerente, necesitas hacer un `JOIN` costoso.



id	nombre	salario	rol	bonificación_gerente	lenguajes_programador	depto	fecha_inicio	es_gerente	es_programador
1	nombre 1	salario	rol	NULL	N/A	depto	07/01/2021	True	NULL
2	nombre 2	salario	gerente	NULL	N/A	depto	07/01/2021	True	N/A
3	nombre 3	salario	employee	NULL	N/A	depto	07/01/2021	True	N/A
4	nombre 4	salario	empleado	NULL	N/A	depto	07/01/2021	True	N/A
5	nombre 5	salario	programador	NULL	N/A	depto	07/07/2021	True	NULL
6	nombre 6	salario	programador	NULL	N/A	depto	07/07/2021	True	N/A
7	nombre 7	salario	employer	NULL	N/A	esnrente	06/07/2021	True	NULL
8	nombre 8	salario	gerente	NULL	N/A	depto	06/07/2021	True	NULL
9	nombre 9	salario	employee	NULL	N/A	depto	07/07/2021	True	NULL
10	nombre 10	salario	gerente	NULL	N/A	depto	07/07/2021	True	N/A
11	nombre 11	salario	gerente	NULL	N/A	depto	07/07/2021	True	NULL
12	nombre 12	salario	programador	NULL	N/A	depto	07/07/2021	True	NULL
13	nombre 13	salario	employee	NULL	N/A	depto	07/07/2021	True	NULL

La Naturalidad en BDOO

En una base de datos de objetos, **la base de datos entiende la jerarquía**.

- Tú guardas un objeto `Programador`.
- La BDOO sabe que es también un `Empleado`.
- No hay tablas separadas ni JOINs. Los datos se guardan juntos en el objeto.
- Si haces una consulta: `SELECT * FROM Empleado`, la base de datos te devolverá automáticamente tanto objetos `Empleado`, como `Programador` y `Gerente`.

2. Soporte para Polimorfismo

El polimorfismo es la capacidad de tratar objetos de diferentes tipos de forma uniforme.

Imagina que tienes una lista en Java:

Java
```
List<Empleado> plantilla = new ArrayList<>();
plantilla.add(new Programador("Ana"));
plantilla.add(new Gerente("Luis"));
```

En una BDOO:

- Puedes guardar esa lista `plantilla` directamente.
- Cuando recuperes la lista de la base de datos y la recorras:
    - El primer objeto será una instancia real de la clase `Programador`.
    - El segundo objeto será una instancia real de la clase `Gerente`.
- El operador `instanceof` funciona perfectamente tras recuperar los datos.

En una base de datos relacional, al recuperar datos de una tabla, todo son "filas". Para reconstruir el objeto correcto, necesitas columnas discriminadoras (tipo `"TIPO_EMPLEADO" = "GER"`) y lógica compleja en tu código para instanciar la clase correcta manualmente.

3. Encapsulación y Visibilidad

La encapsulación dicta que los datos internos de un objeto (`private`) no deben ser accesibles desde fuera, sino a través de métodos.

- **El problema Relacional:** Una tabla es pública. Cualquier DBA con acceso a la consola SQL puede hacer un `UPDATE` en la columna `salario` y romper la lógica de negocio.
- **La visión BDOO:**
    - Las BDOO respetan la visibilidad de Java.
    - Son capaces de guardar y recuperar atributos **`private`** directamente (usando reflexión interna) sin obligarte a crear Getters y Setters públicos que rompan tu encapsulación solo para la persistencia.
    - Los datos y el comportamiento viajan juntos. Algunas BDOO incluso permiten ejecutar métodos de la clase dentro de la consulta (aunque esto varía según el motor).

4. Relaciones (Asociaciones)

Olvídate de las **Claves Foráneas (Foreign Keys)** y de las tablas intermedias para relaciones Muchos-a-Muchos.

En una BDOO, una relación es simplemente una referencia.

- **Relación 1:1:** Un atributo de tipo objeto (`private Direccion direccion;`).
- **Relación 1:N:** Una colección (`private List<Factura> facturas;`).

La navegación es directa. No escribes `SELECT ... JOIN ... ON ...`. Simplemente accedes a la referencia:

Java
```
// Navegación en BDOO (Java puro)
String calle = cliente.getDireccion().getCalle();
```

La base de datos carga el objeto relacionado de forma transparente (y generalmente perezosa/lazy) cuando lo pides.

3. Instalación del gestor de BDOO

3.1. Elección del motor: Gestores populares (db4o, ObjectDB, Perst)

El mercado de las BDOO es más de nicho que el de las relacionales. No hay cientos de opciones, pero las que hay son muy especializadas. Para este curso (y para Java en general), los tres grandes nombres que debes conocer son:

1. ObjectDB (El Líder Moderno)

Es, con diferencia, la BDOO más potente y utilizada hoy en día en el ecosistema Java empresarial.

- **Filosofía:** Rendimiento extremo. Se jacta de ser más rápida que cualquier base de datos relacional con Hibernate.
- **Estándares:** Es 100% compatible con **JPA (Java Persistence API)** y **JDO**. Esto es una ventaja enorme: si sabes usar Hibernate/JPA, ya sabes usar ObjectDB.
- **Licencia:** Propietaria, pero gratuita para desarrollo, educación y uso con ciertas limitaciones de tráfico.
- **Uso recomendado:** Proyectos profesionales nuevos que requieran velocidad máxima y compatibilidad con estándares Java EE.

2. db4o (Database for Objects - El Clásico Educativo)

Fue la base de datos que popularizó el concepto de "Persistencia Nativa" en los 2000.

- **Filosofía:** Simplicidad pura. Su API es minimalista (`db.store(objeto)`). No usa JPA, sino su propia forma de trabajar.
- **Estado:** Fue adquirida por Versant y su desarrollo se detuvo hace años. Aunque está considerada "legacy" (obsoleta para producción masiva moderna), sigue siendo **la mejor herramienta pedagógica** para entender cómo funciona una BDOO pura sin la complejidad de JPA.
- **Licencia:** Open Source (GPL) y Comercial.
- **Uso recomendado:** Aprendizaje, prototipos rápidos y sistemas embebidos antiguos.

3. Perst (El Ligero)

Es una base de datos orientada a objetos de código abierto diseñada para tener una huella de memoria muy pequeña.

- **Filosofía:** Compacta y rápida. Diseñada específicamente para dispositivos móviles, IoT y aplicaciones de tiempo real.
- **Características:** No requiere administración. Soporta cifrado y replicación.
- **Licencia:** Open Source (GPL) y Comercial.
- **Uso recomendado:** Aplicaciones Android, dispositivos IoT o aplicaciones de escritorio ligeras.

Tabla Comparativa para la Elección

Característica	ObjectDB	db4o	Perst
Tipo	BDOO Híbrida (soporta JPA)	BDOO Pura	BDOO Embebida Pura
API Principal	**JPA / JDO** (Estándar)	**SODA / Nativa** (Propietaria)	API Propietaria
Rendimiento	Extremadamente Alto	Alto	Muy Alto (en local)

Característica	ObjectDB	db4o	Perst
Curva de Aprendizaje	Media (requiere saber JPA)	**Muy Baja** (facilísimo)	Media
Estado Actual	Activo y mantenido	Inactivo / Legacy	Mantenimiento bajo
Instalación	1 archivo `.jar`	1 archivo `.jar`	1 archivo `.jar`

¿Cuál vamos a usar en este curso?

Para aprender los conceptos **fundamentales** de las bases de datos orientadas a objetos (sin distraernos con la complejidad de JPA, que veremos más adelante), **db4o** suele ser la elección didáctica favorita por su simplicidad.

Sin embargo, si buscas una salida profesional inmediata, **ObjectDB** es la tecnología que usarías en una empresa real.

> **Mi recomendación:** Vamos a explicar la configuración basándonos en **ObjectDB** o **db4o** (dado que la sintaxis de db4o es más "pura" para los ejemplos de OQL y QBE de esta unidad, suele ser la elegida en los temarios académicos clásicos de FP y Universidad).

3.2. Configuración del entorno: Importación de librerías JAR

Para que tu aplicación Java pueda crear y gestionar una base de datos orientada a objetos, necesita las clases que saben cómo hacerlo. Como estas clases no vienen en el JDK estándar (como sí venía JDBC), debemos añadir una librería externa.

El proceso es idéntico al de añadir un Driver JDBC, pero aquí esa librería no es solo un conector, **es la base de datos completa**.

Paso 1: Obtener el fichero `.jar`

Dependiendo del motor que hayas elegido (ObjectDB o db4o), deberás ir a su web oficial y descargar el paquete de distribución.

- **Para ObjectDB:** Descargas un `.zip` que contiene `objectdb.jar`.
- **Para db4o:** Descargas una distribución que contiene `db4o-8.x-java5.jar` (o similar).

Paso 2: Importación Manual (Proyecto Básico)

Si estás aprendiendo y no usas gestores de dependencias como Maven o Gradle, este es el método clásico.

1. **Crea una carpeta de librerías:** En la raíz de tu proyecto Java (en Eclipse, IntelliJ o NetBeans), crea una carpeta llamada `lib` o `libs`.
2. **Copia el JAR:** Pega dentro el archivo `.jar` que descargaste.

3. **Añadir al Classpath (Build Path):** Este es el paso vital. No basta con que el archivo esté en la carpeta; debes decirle al IDE que lo use para compilar.

- **En IntelliJ IDEA:**
  - Clic derecho sobre el `.jar` dentro de la carpeta `lib`.
  - Seleccionar opción **"Add as Library..."**.
- **En Eclipse:**
  - Clic derecho sobre el proyecto -> *Build Path* -> *Configure Build Path*.
  - Pestaña *Libraries* -> *Classpath* -> botón *Add JARs...* y seleccionas tu archivo.

Paso 3: Importación Profesional (Maven)

Si estás en un entorno profesional, usarás Maven. Solo necesitas añadir la dependencia en tu `pom.xml`.

Ejemplo para ObjectDB: Como ObjectDB no está en el repositorio central de Maven (por temas de licencia), debes añadir su repositorio propio primero.

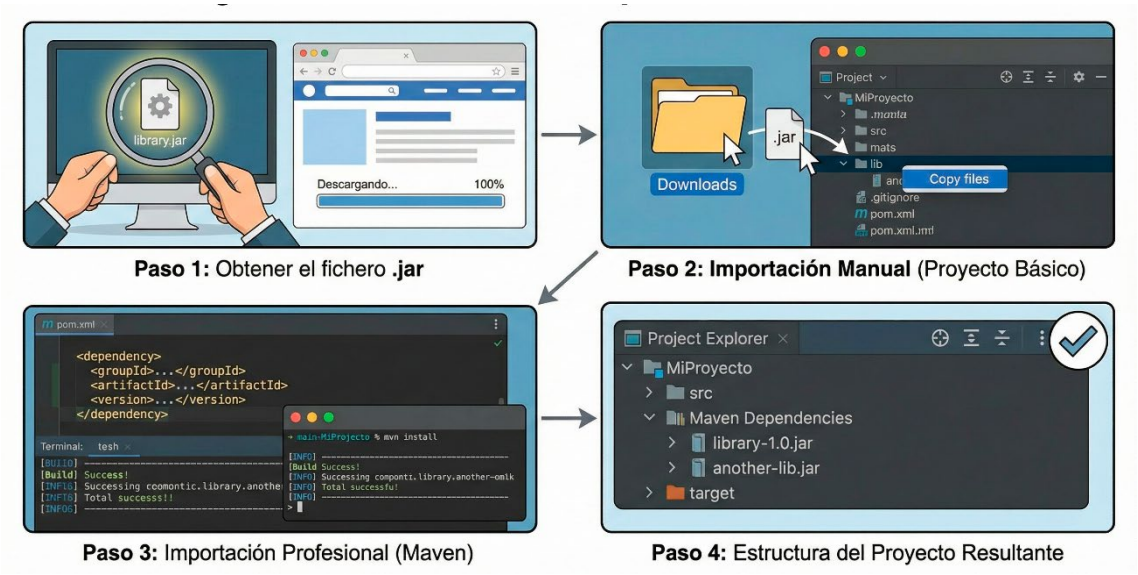
XML
```xml
<repositories>
    <repository>
        <id>objectdb</id>
        <name>ObjectDB Repository</name>
        <url>https://m2.objectdb.com</url>
    </repository>
</repositories>

<dependencies>
    <dependency>
        <groupId>com.objectdb</groupId>
        <artifactId>objectdb</artifactId>
        <version>2.8.1</version>
    </dependency>
</dependencies>
```

Paso 4: Estructura del Proyecto Resultante

Una vez configurado, tu proyecto tendrá una estructura similar a esta. Fíjate en dónde aparecerá el fichero de la base de datos (normalmente `.odb` o `.wap`) una vez ejecutes el programa:

Plaintext
```
MiProyectoJava/
├── src/
│   └── main/
│       └── java/
│           └── com/empresa/app/
│               ├── Main.class
│               └── Modelo/
│                   ├── Cliente.java
│                   └── Factura.java
├── lib/
│   └── objectdb.jar  <-- El motor de la BDOO
├── target/
├── pom.xml
└── mi_base_datos.odb <-- ¡Aquí se guardarán tus datos!
```



Importante: Observa que el fichero de datos (`mi_base_datos.odb`) aparece como un archivo más dentro de tu carpeta de proyecto. **No hay un servidor externo**. Si borras ese archivo, borras la base de datos. Si quieres hacer una copia de seguridad, solo tienes que copiar y pegar ese archivo. Así de simple.

3.3. Modos de funcionamiento: Servidor vs. Embebida

Cuando configuras tu base de datos de objetos en el código, debes tomar una decisión arquitectónica: ¿Quieres que la base de datos viva **dentro** de tu programa o que viva **fuera**?

1. Modo Embebido (Embedded Mode)

Es el modo "natural" y por defecto de estas bases de datos.

- **Cómo funciona:** El motor de la base de datos se ejecuta **dentro del mismo proceso (JVM)** que tu aplicación. No hay un proceso de base de datos separado en el Administrador de Tareas. Tu aplicación y la base de datos son uno solo.
- **Comunicación:** Las llamadas son directas en memoria. Cuando pides un objeto, no hay cables de red, ni sockets, ni serialización TCP/IP. Es una llamada a un método local.
- **El Archivo de Datos:** Tu aplicación obtiene acceso exclusivo de lectura/escritura al fichero `.odb` o `.wap`.

Ventajas:

- **Velocidad extrema:** Es la forma más rápida de persistencia que existe. Cero latencia de red.
- **Administración cero:** No hay que instalar servicios ni crear usuarios.
- **Despliegue fácil:** Tu aplicación es portable (un .jar y un archivo de datos).

Desventajas:

- **Bloqueo de archivo:** Como tu aplicación tiene el archivo "secuestrado", **ninguna otra aplicación** puede acceder a la base de datos al mismo tiempo. Es monousuario/monoaplicación.

2. Modo Cliente-Servidor (Client-Server Mode)

Es el modo clásico que conocemos de MySQL u Oracle.

- **Cómo funciona:**

1. Ejecutas un programa Java separado (que usa el mismo `.jar`) que actúa como **Servidor**. Este programa se queda escuchando en un puerto (ej: 6136).
2. Tu aplicación (y la de otros usuarios) actúa como **Cliente** y se conecta a ese puerto mediante la red (TCP/IP).

    - **Comunicación:** Los objetos viajan por la red.
    - **El Archivo de Datos:** Solo el proceso Servidor toca el archivo físico. Los clientes le piden cosas al Servidor.

Ventajas:

- **Concurrencia:** Múltiples aplicaciones o múltiples usuarios pueden leer y escribir datos a la vez.
- **Centralización:** Los datos están en un lugar seguro, no dispersos en los ordenadores de los usuarios.

Desventajas:

- **Latencia:** Es más lento que el modo embebido debido al tráfico de red.
- **Complejidad:** Necesitas asegurar que el servidor esté siempre encendido.

Comparativa y Código

Lo maravilloso de las BDOO es que cambiar de un modo a otro suele requerir cambiar **una sola línea de código**.

Ejemplo con db4o / ObjectDB

A. Código para Modo Embebido (Local): Le pasas una ruta de archivo.

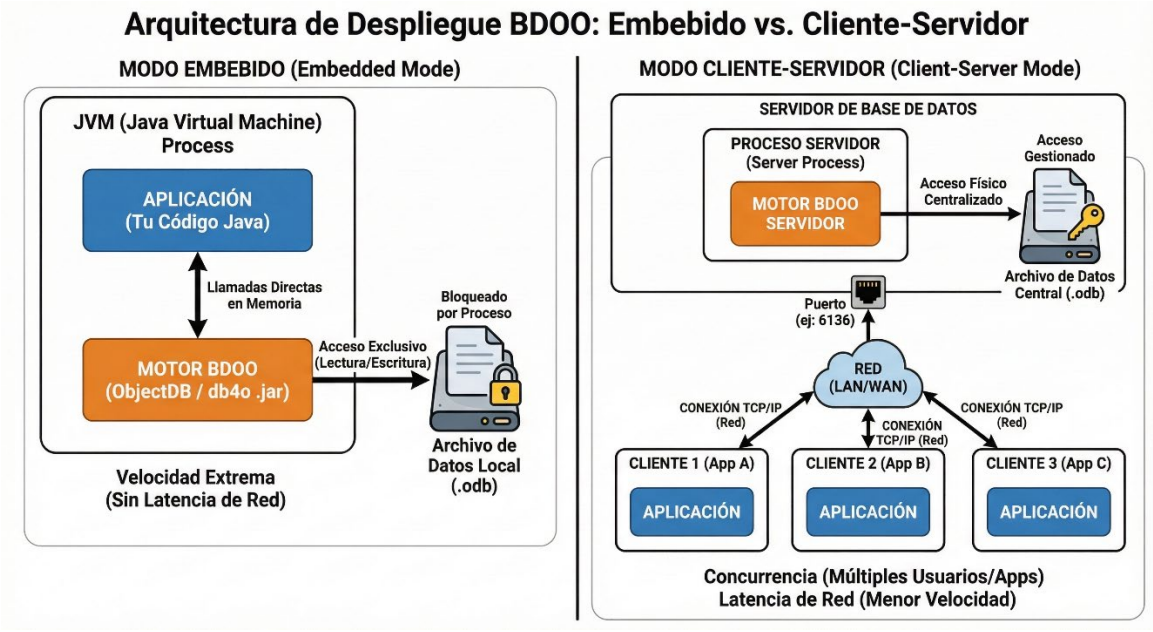
Java
```
// Abres el archivo directamente.
// Si el archivo está bloqueado por otro programa, esto fallará.
ObjectContainer db = Db4oEmbedded.openFile("C:/datos/miBase.wap");
```

B. Código para Modo Cliente-Servidor (Remoto): Le pasas una IP, puerto, usuario y contraseña.

Java
```
// Te conectas a alguien que ya tiene el archivo abierto
ObjectContainer db = Db4oClient.openClient("localhost", 6136,
"user", "pass");
```

¿Cuándo usar cada uno?

Escenario	Modo Recomendado
App Móvil (Android)	**Embebido**. Cada móvil tiene sus datos locales.
App de Escritorio Monousuario	**Embebido**. (Ej: Una agenda personal).
Web Service / Backend	**Embebido**. (Si solo hay un servidor web accediendo a los datos).
Sistema Empresarial Distribuido	**Cliente-Servidor**. Varios departamentos accediendo a los mismos datos.
Desarrollo y Pruebas	**Embebido**. Para no complicarse configurando servidores.



4. Creación de bases de datos

4.1. El contenedor de objetos: Apertura y cierre de sesiones (`ObjectContainer`)

En las bases de datos orientadas a objetos, el protagonista absoluto es el **Contenedor de Objetos** (en la terminología de *db4o* se llama interfaz **`ObjectContainer`**; en JPA/ObjectDB se llama `EntityManager`).

Este objeto cumple la doble función de:

1. **Base de Datos:** Representa el archivo físico en el disco.
2. **Sesión/Conexión:** Representa tu transacción activa. Mientras tengas el contenedor abierto, puedes guardar y leer.

1. Apertura (y Creación Automática)

En el mundo SQL, antes de conectar, solías necesitar un administrador que ejecutara `CREATE DATABASE mi_db`.

En las BDOO embebidas, rige el principio de **"Creación bajo demanda"**:

- Tú pides abrir un archivo de base de datos.
- Si el archivo **ya existe**, el motor lo abre y te da acceso.
- Si el archivo **no existe**, el motor **lo crea automáticamente** en ese mismo instante y te da acceso.

No existe un comando `CREATE DATABASE` explícito. El simple acto de conectarse la crea.

Sintaxis (db4o):
Java
```
import com.db4o.Db4oEmbedded;
import com.db4o.ObjectContainer;

// Ruta al archivo físico
String rutaArchivo = "C:/datos/miBase.wap";

// Al ejecutar esto, si 'miBase.wap' no existe, SE CREA.
ObjectContainer db = Db4oEmbedded.openFile(rutaArchivo);
```

2. El Ciclo de Vida de la Sesión

El `ObjectContainer` funciona igual que la `Connection` de JDBC que vimos en la Unidad 9. Es un recurso valioso que consume memoria y bloquea el archivo para que otros no lo toquen.

Por tanto, el ciclo de vida obligatorio es:

1. **Abrir** (`openFile`).
2. **Operar** (Guardar, buscar, borrar objetos).
3. **Cerrar** (`close`).

3. Cierre de Sesiones

Es **crítico** cerrar el contenedor antes de que termine tu programa.

- Si no cierras (`db.close()`), los datos que están en la memoria caché (RAM) podrían no escribirse en el disco duro.
- El archivo podría quedar "corrupto" o bloqueado, impidiendo que lo abras la próxima vez.

El método antiguo (Java 6 e inferiores): Usábamos un bloque `finally` para asegurar el cierre.

Java
```
ObjectContainer db = null;
try {
    db = Db4oEmbedded.openFile("base.wap");
    // ... operaciones ...
} finally {
```

```
    if (db != null) {
        db.close(); // ¡Obligatorio!
    }
}
```

El método moderno (Try-with-resources): Al igual que en JDBC, los contenedores modernos implementan AutoCloseable, por lo que podemos dejar que Java lo cierre solo.

Java
```
// Código Recomendado
try (ObjectContainer db = Db4oEmbedded.openFile("base.wap")) {

    System.out.println("Base de datos abierta/creada con éxito.");
    // Aquí haremos las operaciones en los siguientes puntos...

} // Al llegar a esta llave, se ejecuta db.close() automáticamente.
```

Comparativa: JDBC vs. BDOO

Para que te sitúes mentalmente, aquí tienes la equivalencia de conceptos:

Concepto	En JDBC (SQL)	En BDOO (db4o / ObjectDB)
Objeto Principal	`Connection`	`ObjectContainer`
Crear la DB	Comando SQL `CREATE DATABASE`	Automático al abrir (`openFile`)
Transacción	`conn.commit()` manual	Automático al cerrar (o `db.commit()`)
Cierre	`conn.close()`	`db.close()`

4.2. Estructura física: Ficheros de datos y gestión del almacenamiento

A diferencia de los grandes servidores SQL (como Oracle), que gestionan particiones, *tablespaces*, archivos de log y configuraciones complejas de sistema de archivos, las BDOO embebidas suelen apostar por la **simplicidad monolítica**.

1. El Fichero Único (The Single Container)

En la inmensa mayoría de los casos (db4o, ObjectDB), toda la base de datos reside en **un único fichero físico** en el disco duro.

- **Extensión:** Suelen ser `.odb` (ObjectDB), `.yap` o `.wap` (db4o), aunque puedes ponerle la extensión que quieras.
- **Contenido:** Dentro de ese archivo conviven:
    - Los metadatos (definiciones de clases).

- Los índices (B-Trees para búsquedas rápidas).
- Los objetos serializados (los datos en sí).
- El gestor de espacio libre.

Ventaja: Hacer una copia de seguridad (Backup) es tan trivial como copiar ese archivo y pegarlo en un USB. No necesitas herramientas de "Exportación/Importación".

2. Direccionamiento Físico (El ID como Puntero)

Aquí reside el secreto de la velocidad de las BDOO.

En una base de datos relacional, para encontrar una fila, el motor busca en un índice la Clave Primaria, obtiene una dirección y luego va al disco.

En muchas BDOO, el **OID (Identificador del Objeto)** es, literalmente, la **dirección de memoria física (offset)** del objeto dentro del archivo.

- Si el sistema te dice que el objeto tiene ID `10240`, significa que el objeto empieza exactamente en el **byte 10.240** del archivo.
- **Acceso O(1):** El disco duro salta directamente a esa posición y lee. No hay búsquedas intermedias. Es el mecanismo de acceso más rápido posible.

3. El ciclo de vida del almacenamiento: Inserción y Modificación

¿Qué ocurre físicamente cuando guardas o cambias datos?

A. Inserción (Store)

Cuando guardas un objeto nuevo, el motor busca espacio libre (generalmente al final del archivo), serializa el objeto (lo convierte a bytes) y lo escribe allí. Devuelve la dirección de inicio como su nuevo ID.

B. Modificación (Update) - El problema del tamaño variable

Aquí es donde se complica. Imagina que tienes un objeto `Usuario` que ocupa 100 bytes. Lo modificas añadiendo un texto largo en su biografía y ahora ocupa 150 bytes.

El objeto ya no cabe en su hueco original.

1. El motor marca el hueco original de 100 bytes como "Espacio Libre" (hueco o *gap*).
2. Escribe el objeto actualizado (150 bytes) al final del archivo.
3. Actualiza internamente la referencia para que apunte a la nueva dirección.

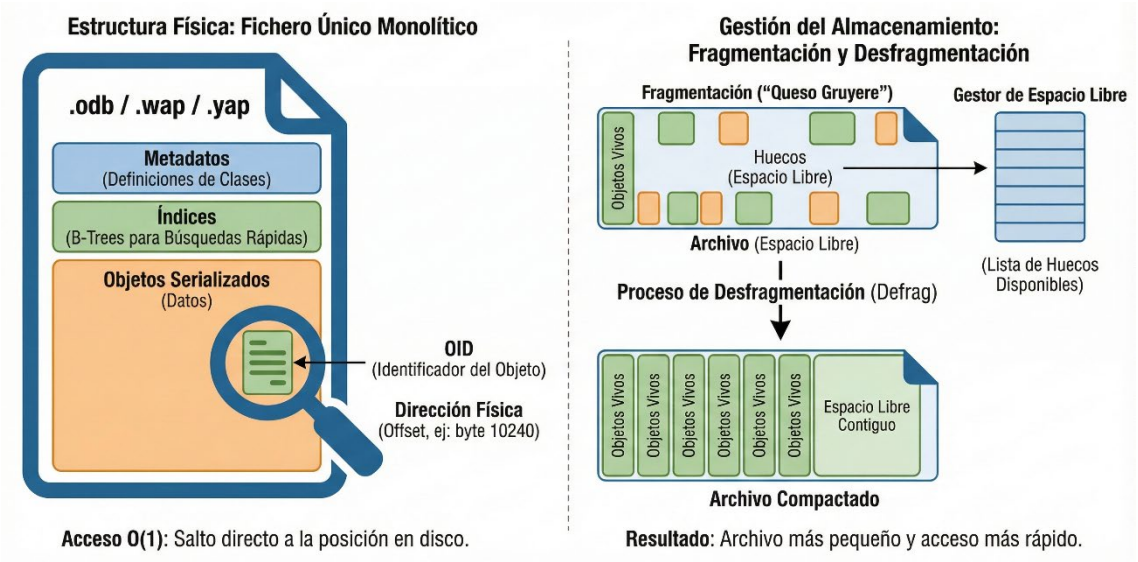
4. Gestión del Espacio y Fragmentación

Debido al proceso de actualización y borrado, el archivo de la base de datos empieza a parecerse a un **queso gruyere**. Se llena de huecos vacíos donde antes había objetos.

Para gestionar esto, las BDOO tienen dos mecanismos:

1. **El Gestor de Espacio Libre (Free Space Manager):** El sistema mantiene una lista interna de "huecos disponibles". Si vas a guardar un objeto pequeño, el sistema intenta rellenar uno de esos huecos antiguos en lugar de hacer crecer el archivo.
2. **Desfragmentación (Defrag):** Con el tiempo, el archivo puede estar muy fragmentado (mucho espacio desperdiciado). Los gestores incluyen una utilidad de Defragment que:

   - Crea un archivo nuevo.
   - Lee todos los objetos vivos del archivo viejo y los copia contiguamente en el nuevo.
   - Elimina los huecos.
   - El archivo resultante es más pequeño y más rápido.



5. Seguridad Física: Transacciones ACID en un fichero

Si se va la luz mientras el disco duro está escribiendo en el archivo, la base de datos podría corromperse irremediablemente. Para evitar esto, se usan técnicas como **Shadow Copy** o **Write-Ahead Logging**.

Ejemplo de Seguridad (Shadow Copy): Cuando haces commit():

1. La base de datos **no sobrescribe** los datos antiguos todavía.
2. Escribe los nuevos datos en una zona libre.
3. Solo cuando los nuevos datos están escritos y verificados completamente, se actualiza un puntero maestro en la cabecera del archivo.
4. Si la luz se va a mitad del paso 2, al reiniciar, el puntero maestro sigue apuntando a los datos antiguos. No se ha perdido nada (Atomicidad).

4.3. Configuración inicial: Índices, cascada y profundidad

Las bases de datos orientadas a objetos intentan ser transparentes, pero no son mágicas. Si guardas un objeto que tiene referencias a otros mil objetos, la base de datos necesita saber: "¿Quieres que guarde también los mil hijos? ¿Y si leo el padre, cargo los mil hijos en la RAM?".

Para controlar esto, ajustamos tres parámetros vitales.

1. Índices (Performance)

Por defecto, guardar objetos es rápido, pero buscarlos puede ser lento. Si buscas `Usuario` por su "nombre" y no hay índice, la BDOO tiene que abrir **todos** los objetos Usuario uno por uno para ver cómo se llaman (Escaneo Secuencial).

Debemos indicar explícitamente qué atributos se usarán frecuentemente en las búsquedas (`WHERE`).

- **Sin Índice:** O(n). Si tienes 1 millón de usuarios, hace 1 millón de comparaciones.
- **Con Índice:** O(log n). El sistema crea un árbol interno (B-Tree). La búsqueda es instantánea.

Ejemplo de configuración (db4o):

Java
```
EmbeddedConfiguration config = Db4oEmbedded.newConfiguration();

// Indicamos: En la clase 'Usuario', indexa el campo 'nombre'
config.common().objectClass(Usuario.class).objectField("nombre").in
dexed(true);

// Ahora abrimos la base de datos con esta configuración
ObjectContainer db = Db4oEmbedded.openFile(config, "base.wap");
```

2. Activación de Cascada (Integridad Referencial)

Aquí definimos qué pasa con los objetos relacionados cuando hacemos una operación de escritura (`store` o `delete`).

A. Cascada en Actualización (Cascade On Update)

Imagina que tienes un objeto `Factura` que contiene un objeto `Cliente`.

- Cambias el teléfono del cliente en memoria.
- Haces `db.store(factura)`.
- **Por defecto:** La base de datos guarda la factura, pero **NO** actualiza los datos del cliente (para ahorrar tiempo). El cambio del teléfono se pierde.
- **Configuración:** Debes activar `cascadeOnUpdate` si quieres que al guardar al padre, se guarden los hijos modificados.

B. Cascada en Borrado (Cascade On Delete)

- Haces `db.delete(factura)`.

- **Por defecto:** Se borra la factura, pero el cliente se queda en la base de datos (quizás huérfano).
- **Configuración:** Si activas `cascadeOnDelete`, al borrar la factura se borra el cliente. **¡Cuidado!** Si ese cliente tenía otras facturas, acabas de romper la base de datos. Úsalo con precaución (ej: borrar un Pedido debe borrar sus Líneas de Pedido, pero no sus Productos).

Java
```
// Ejemplo: Al guardar una Factura, revisa y guarda cambios en sus
hijos
config.common().objectClass(Factura.class).cascadeOnUpdate(true);
```

3. Niveles de Profundidad de Activación (Activation Depth)

Este es el concepto **más importante** y confuso de las BDOO. Se refiere a la **lectura (SELECT)**.

Cuando pides un objeto a la base de datos, el motor tiene que "instanciarlo" en la memoria RAM. A esto se le llama Activar. El problema es que los objetos forman una red infinita: `Coche -> Motor -> Tornillo -> Fabricante -> ListaDeClientes`…

Si la base de datos cargara todo lo que está conectado, cargaría la base de datos entera en la RAM y tu ordenador explotaría (**Out of Memory**).

La Solución: Profundidad de Activación (Lazy Loading)

La base de datos solo carga hasta cierto nivel de profundidad.

- **Nivel 0:** El objeto que pediste.
- **Nivel 1:** Los objetos directos dentro de él.
- **Nivel 5 (Por defecto en db4o):** Hasta 5 saltos de referencia.

¿Qué pasa más allá del límite? Los atributos aparecen como `null` (o desactivados) en tu código Java, aunque en la base de datos sí existan datos.

- *Síntoma típico:* Haces `coche.getMotor().getTornillo().getFabricante()` y te da un `NullPointerException`, pero jurarías que guardaste el fabricante.
- *Causa:* La "profundidad de activación" por defecto no llegó tan lejos.

Cómo solucionarlo:

1. **Aumentar la profundidad global:** (Peligroso para el rendimiento).
2. **Configuración específica por clase:** "Para la clase Coche, activa 10 niveles".
3. **Activación manual:** Decirle a la base de datos explícitamente "Activa este objeto ahora".

Java
```
// Configuración global: Bajar la profundidad a 2 para ahorrar
memoria
config.common().activationDepth(2);

// Configuración específica: La clase 'ArbolGenealogico' necesita
mucha profundidad
config.common().objectClass(ArbolGenealogico.class).minimumActivati
onDepth(10);
```

5. Mecanismos de consulta

5.1. Consultas por Ejemplo (QBE - Query by Example)

Query by Example (QBE) es una técnica de búsqueda basada en **prototipos**.

En lugar de aprender un lenguaje nuevo (como SQL) y escribir cadenas de texto complejas (`"SELECT * FROM..."`), en QBE simplemente creas un objeto de la clase que quieres buscar, rellenas los datos que conoces y dejas vacíos los que no te importan. Luego le dices a la base de datos: *"Dame todo lo que se parezca a esto"*.

Es como ir a una tienda de ropa, enseñar una camisa azul y decir: "¿Tenéis más como esta?".

1. ¿Cómo funciona el "Prototipo"?

El mecanismo se basa en una regla simple de coincidencia (matching):

- **Campos con valor (`!= null`):** Actúan como **Filtro**. La base de datos buscará objetos que tengan *exactamente* ese valor.
- **Campos vacíos (`null` o `0`):** Actúan como **Comodín (Wildcard)**. La base de datos ignorará estos campos (significa "no me importa lo que haya aquí").

2. Ejemplos Prácticos (Sintaxis db4o)

Imaginemos que tenemos una clase `Piloto` con los atributos: `nombre` (String), `puntos` (int) y `equipo` (String).

A. Recuperar TODOS los objetos (Select All)

Si pasas un prototipo vacío (o `null` en todos sus campos), la base de datos entiende que no hay restricciones.

Java
```
// 1. Creamos el prototipo vacío (todo a null/0)
Piloto prototipo = new Piloto(null, 0, null);

// 2. Ejecutamos la consulta
ObjectSet<Piloto> resultado = db.queryByExample(prototipo);

// 3. Resultado: Devuelve TODOS los pilotos de la base de datos.
```

B. Filtrar por un atributo exacto

Queremos encontrar a todos los pilotos del equipo "Ferrari".

Java
```
// 1. Rellenamos SOLO el campo que nos interesa
Piloto prototipo = new Piloto();
prototipo.setEquipo("Ferrari");
// nombre es null, puntos es 0 -> Se ignoran
```

```
// 2. Ejecutamos
ObjectSet<Piloto> resultado = db.queryByExample(prototipo);

// 3. Resultado: Lista de pilotos de Ferrari, tengan los puntos que
tengan.
```

C. Filtrar por múltiples atributos (AND implícito)

Queremos encontrar al piloto "Fernando Alonso" que corra en "Aston Martin".

Java
```
Piloto prototipo = new Piloto();
prototipo.setNombre("Fernando Alonso");
prototipo.setEquipo("Aston Martin");

ObjectSet<Piloto> resultado = db.queryByExample(prototipo);
// Devuelve el objeto solo si coinciden AMBOS campos (Lógica AND).
```

3. Limitaciones Graves de QBE

Aunque QBE es genial para empezar porque no requiere aprender sintaxis nueva, es **muy limitado** para aplicaciones reales complejas. Tiene tres problemas fundamentales:

1. El problema de los Tipos Primitivos (El "Cero")

En Java, un `int` nunca puede ser `null`; por defecto vale `0`.

- Si buscas un piloto con `puntos = 0`, QBE se confunde: ¿Estás buscando pilotos con 0 puntos o simplemente dejaste el campo por defecto porque no te importan los puntos?
- **Solución:** Usar clases envolventes (`Integer` en vez de `int`) para poder usar `null`, pero esto obliga a cambiar tu modelo de datos.

2. Falta de Operadores Avanzados

QBE solo entiende de **igualdad exacta** (`==`).

- No puedes buscar: "Pilotos con **más de** 100 puntos" (`>`).
- No puedes buscar: "Pilotos cuyo nombre **contenga** 'Schu'" (`LIKE`).
- No puedes usar `OR` ("Pilotos de Ferrari **O** de Red Bull").

3. Profundidad

Si tu objeto prototipo tiene otros objetos dentro, QBE intentará buscar coincidencias exactas en los objetos anidados, lo cual puede complicarse si no tienes cuidado con la configuración.

Resumen de QBE

Ventajas	Desventajas
Sencillez extrema: Cualquiera sabe usarlo en 1 minuto.	**Solo igualdad:** No admite `>`, `<`, `LIKE`, `OR`.
Refactorización: Si cambias el nombre de un atributo en la clase, la consulta se actualiza sola (no son Strings).	**Problema del 0:** Confusión con valores por defecto en primitivos.
Sin sintaxis extra: Es 100% Java.	**Poca potencia:** Inútil para informes complejos.

5.2. Consultas Nativas (Native Queries)

Las Consultas Nativas permiten expresar criterios de búsqueda complejos utilizando código Java estándar.

La filosofía es simple: **Si sabes programar un `if` en Java, sabes hacer una consulta.**

1. ¿Cómo funcionan?

En lugar de construir una cadena de texto (como en SQL o OQL), tú escribes una pequeña clase o función que devuelve `true` o `false`.

La base de datos toma todos los objetos candidatos y los pasa uno por uno por tu función:

- Si tu función devuelve **`true`**, el objeto se incluye en el resultado.
- Si devuelve **`false`**, se descarta.

2. La interfaz `Predicate`

En **db4o** (el referente principal de esta tecnología), las consultas nativas se implementan mediante la clase abstracta `Predicate`. Debes sobrescribir su método `match()`.

Sintaxis:
Java
```
List<Clase> resultado = db.query(new Predicate<Clase>() {
    public boolean match(Clase candidato) {
        // Tu lógica Java aquí (return true/false)
        return ...;
    }
});
```

3. Ejemplo Práctico: Superando las limitaciones de QBE

Retomemos el ejemplo de los **Pilotos**. Con QBE no podíamos buscar rangos ni cadenas parciales. Con Native Queries, sí.

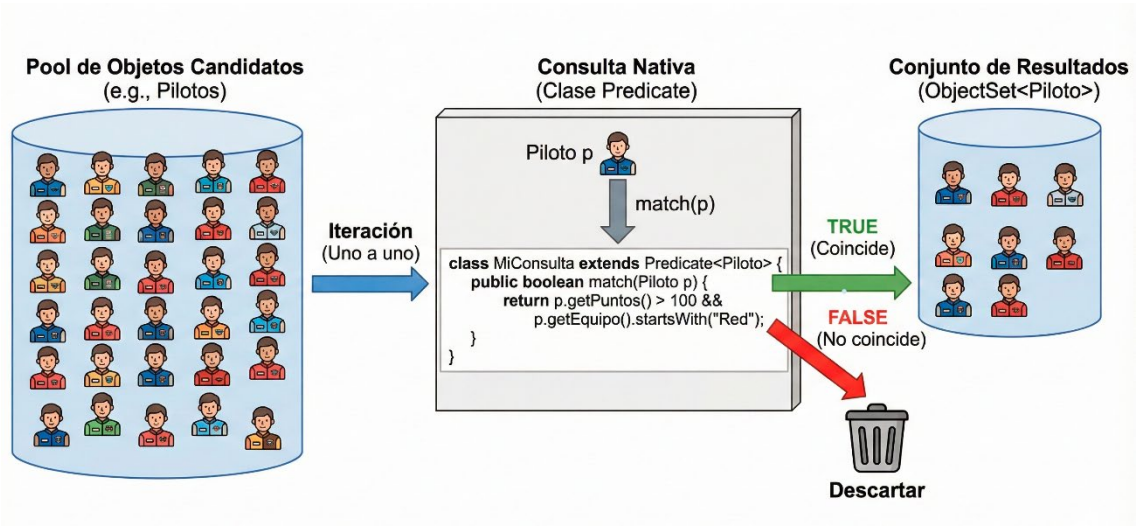
Objetivo: Buscar pilotos que tengan **más de 100 puntos** Y cuyo equipo empiece por **"Red"**.

Java
```java
import com.db4o.query.Predicate;
// ...

ObjectSet<Piloto> result = db.query(new Predicate<Piloto>() {
    @Override
    public boolean match(Piloto p) {
        // ¡Aquí tienes toda la potencia de Java!
        // 1. Condición numérica (Range)
        boolean puntosSuficientes = p.getPuntos() > 100;

        // 2. Condición de texto (String operations)
        boolean esEquipoRed = p.getEquipo().startsWith("Red");

        // 3. Lógica Booleana (AND, OR, NOT)
        return puntosSuficientes && esEquipoRed;
    }
});
```



4. Ventajas frente a SQL (y OQL)

1. **Seguridad de Tipos (Type Safety):** Esta es la gran ventaja.

   - En SQL (`"SELECT * FROM Pilotos WHERE puntoss > 100"`), si escribes mal "puntoss", el error salta **cuando ejecutas el programa** (Runtime Error).
   - En Native Queries (`p.getPuntos() > 100`), si escribes mal el método, el IDE te marca rojo y **no te deja compilar**. Detectas el error inmediatamente.

2. **Refactorización:**

   - Si usas Eclipse/IntelliJ para cambiar el nombre del atributo `puntos` a `puntuacion`, el IDE actualiza automáticamente tu consulta Nativa. En SQL, tendrías que buscar y reemplazar texto manualmente.

3. **Potencia:**

- Puedes llamar a métodos complejos dentro de la consulta. Ej: `return p.calcularRendimiento() > 50;`.

5. El problema del rendimiento (Optimización)

Aquí viene la letra pequeña.

Si tienes 1 millón de pilotos, y la base de datos tiene que:

1. Sacar el objeto del disco.
2. Crear la instancia en memoria Java.
3. Ejecutar tu método `match()`. ... un millón de veces, **el sistema será lentísimo**.

La Magia de la Optimización de NQ: Los motores modernos de BDOO (como db4o) intentan ser listos. Cuando escribes tu consulta nativa, el motor **analiza el bytecode** de tu método `match` antes de ejecutarlo.

- Si ve que solo estás haciendo `p.getPuntos() > 100`, traduce eso internamente a una búsqueda de índice rápido (B-Tree).
- Si escribes código muy raro (como conectar a internet dentro del `match`), el motor no puede optimizarlo y tendrá que instanciar todos los objetos (lento).

  Regla de Oro: Mantén las consultas nativas simples para asegurar que la BDOO pueda optimizarlas.

5.3. SODA API (Simple Object Database Access)

SODA es una API de consulta de bajo nivel que permite construir consultas **dinámicamente** en tiempo de ejecución.

¿Recuerdas las Native Queries? Allí escribías el código de filtrado dentro de una clase **antes de compilar.** ¿Pero qué pasa si estás creando un formulario de "¿Búsqueda Avanzada" donde no sabes si el usuario va a filtrar por precio, por nombre, o por ambos hasta que el programa ya está corriendo? Aquí es donde Native Queries falla y SODA brilla.

1. El concepto: El Grafo de Consulta (Query Graph)

SODA no usa strings (SQL) ni clases anónimas (Native). Usa un Grafo de Nodos. Imagínatelo como un árbol invertido:

1. Empiezas por la raíz (la base de datos).
2. Desciendes a la clase (`Piloto`).
3. Desciendes al atributo (`puntos`).
4. Aplicas una restricción (`> 100`).

2. Las palabras clave

Para usar SODA, solo necesitas entender tres métodos del objeto `Query`:

1. `constrain(filtro)`: Significa "restringir". Sirve para decir "Quiero buscar objetos de ESTA clase" o "que valgan ESTO".
2. `descend(campo)`: Significa "bajar". Sirve para navegar desde el objeto padre hacia un atributo hijo (o nieto).
3. `execute()`: Ejecuta el grafo que has construido y devuelve la lista de resultados.

3. Ejemplo Práctico: Construyendo la consulta paso a paso

Vamos a repetir el ejemplo anterior: **"Pilotos con más de 100 puntos y nombre 'Fernando'"**.

Java
```
import com.db4o.query.Query;

// 1. Crear un nodo raíz de consulta
Query query = db.query();

// 2. Restringir a la clase que buscamos (Filtro de Tipo)
query.constrain(Piloto.class);

// 3. Añadir condición 1: Puntos > 100
// Bajamos al campo "puntos"
Query nodoPuntos = query.descend("puntos");
// Aplicamos la restricción y el operador (greater)
nodoPuntos.constrain(100).greater();

// 4. Añadir condición 2: Nombre = "Fernando"
// Bajamos (desde la raíz) al campo "nombre"
query.descend("nombre").constrain("Fernando");

// 5. Ejecutar
ObjectSet<Piloto> result = query.execute();
```

Más info: Fíjate que usamos cadenas de texto (`"puntos"`, `"nombre"`). Aquí perdemos la seguridad de tipos del compilador (si escribes mal el nombre del campo, fallará al ejecutar), pero ganamos flexibilidad total.

4. Operadores y Ordenación

SODA permite hacer todo lo que echábamos de menos en QBE.

- **Comparaciones:** `.greater()`, `.smaller()`, `.equal()`, `.identity()` (por OID).
- **Texto:** `.like()`, `.startsWith()`, `.contains()`.
- **Lógica:** `.and()`, `.or()`, `.not()`.
- **Ordenación:**

Java
```
query.descend("puntos").orderAscending(); // Ordenar por puntos de
menor a mayor
```

5. El "Killer Feature": Búsquedas Dinámicas

¿Por qué usarías SODA en vez de Native Queries? Por esto: Imagina un formulario web donde el usuario puede dejar campos en blanco.

Java

```java
public List<Producto> buscarProductos(String nombre, Double
precioMax) {

    Query q = db.query();
    q.constrain(Producto.class);

    // Solo añadimos el nodo al grafo SI el usuario escribió algo
    if (nombre != null && !nombre.isEmpty()) {
        q.descend("nombre").constrain(nombre).like();
    }

    // Solo añadimos el filtro de precio SI el usuario puso un tope
    if (precioMax != null) {
        q.descend("precio").constrain(precioMax).smaller();
    }

    return q.execute();
}
```

Hacer esto con Native Queries implicaría crear múltiples clases de `Predicate` o llenar el código de `if`s complejos dentro del método `match`. Con SODA, construyes el árbol dinámicamente.

6. Resumen de Mecanismos de Consulta

Ya hemos visto los tres grandes. Aquí tienes cuándo usar cada uno:

Mecanismo	Sintaxis	Ventaja	Inconveniente	Uso Ideal
QBE	Prototipo (`new Piloto(null, 0)`)	Facilísimo.	Muy limitado (sin >, <).	Búsquedas simples exactas.
Native Queries	Java (`p.getPuntos() > 100`)	Seguro (Type Safe) y potente.	Puede ser lento si no optimiza.	Lógica de negocio compleja conocida.
SODA	Grafos (`q.descend("x").constrain(y)`)	El más rápido y dinámico.	Código verborreico y "feo".	Buscadores avanzados y filtros dinámicos.

6. Lenguaje de consultas: Sintaxis, expresiones y operadores

6.1. OQL (Object Query Language): El estándar SQL-like

OQL (Object Query Language) es el lenguaje de consultas estandarizado por el ODMG (Object Data Management Group).

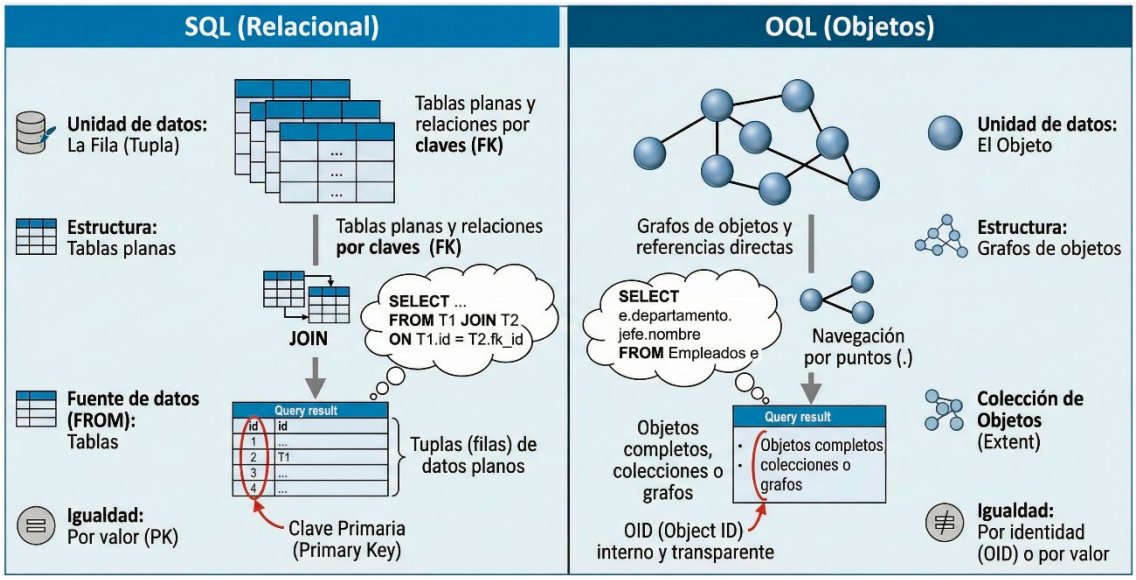
Su filosofía de diseño es brillante: **"Parecerse a SQL todo lo posible para que los programadores no se asusten, pero operar sobre Objetos en lugar de Tablas"**.

Si sabes SQL, ya sabes el 80% de OQL. La diferencia radica en *cómo* navegas por los datos.

1. Diferencias Conceptuales: SQL vs. OQL

Aunque ambos usan palabras clave como `SELECT`, `FROM` y `WHERE`, lo que ocurre "bajo el capó" es muy distinto.

Característica	SQL (Relacional)	OQL (Objetos)
Fuente de datos (`FROM`)	Una o varias **Tablas**.	Una **Colección de Objetos** (Extent) o una Clase.
Relaciones	Explícitas mediante `JOIN ... ON`.	Implícitas mediante **navegación por puntos** (.).
Resultado (`SELECT`)	Tuplas (filas) de datos planos.	Objetos completos, grafos, literales o estructuras.
Igualdad	Por valor (Primary Key).	Por identidad (OID) o por valor.



2. La estructura básica

Una consulta OQL típica tiene esta forma:

SQL
```
SELECT variable
FROM   NombreClase variable
WHERE  condición
```

Fíjate en la cláusula **FROM**.

- En SQL: `FROM empleados` (nombre de la tabla).
- En OQL: `FROM Empleados e` (defines una variable `e` que iterará sobre todos los objetos de la clase `Empleados`).

3. El Superpoder de OQL: Navegación por Rutas (Path Expressions)

Esta es la característica que enamora a los desarrolladores.

Imagina que quieres obtener: *"El nombre de la ciudad donde vive el jefe del departamento en el que trabaja el empleado 'Juan'"*.

En SQL (La pesadilla de los JOINs):
SQL
```
-- SQL Relacional
SELECT ci.nombre
FROM Empleados e
JOIN Departamentos d ON e.id_dep = d.id
JOIN Empleados jefe ON d.id_jefe = jefe.id
JOIN Direcciones dir ON jefe.id_dir = dir.id
JOIN Ciudades ci ON dir.id_ciudad = ci.id
WHERE e.nombre = 'Juan';
```

En OQL (Navegación natural): OQL entiende las relaciones. Si tu clase `Empleado` tiene un atributo `departamento`, simplemente usas un punto.

SQL
```
-- OQL
SELECT e.departamento.jefe.direccion.ciudad.nombre
FROM   Empleados e
WHERE  e.nombre = "Juan"
```

El motor de la base de datos se encarga de "seguir los punteros" automáticamente. No tienes que decirle cómo unir las piezas.

4. Tipos de Retorno

OQL es mucho más flexible que SQL en lo que devuelve.

- **Objetos completos:** `SELECT e FROM Empleados e` (Devuelve una `List<Empleado>`).
- **Atributos sueltos:** `SELECT e.salario FROM Empleados e` (Devuelve una `List<Double>`).
- **Estructuras (Structs):** Puedes crear "objetos al vuelo" con datos mezclados.

SQL
```
SELECT struct(nombre: e.nombre, jefe: e.departamento.jefe.nombre)
FROM Empleados e
```

5. OQL en el mundo real (JPA y JPQL)

Debes saber una curiosidad histórica importante. El estándar ODMG puro (OQL) apenas se usa hoy en día tal cual se definió en los 90.

Sin embargo, su espíritu vive en **JPQL (Java Persistence Query Language)**, que es el lenguaje que usa **Hibernate** y **JPA**. JPQL es, básicamente, una versión moderna de OQL. Por eso, aprender OQL es fundamental: es la base para dominar Hibernate y el acceso a datos en Spring Boot.

> **Resumen:** OQL nos permite escribir consultas textuales potentes que respetan la encapsulación y la herencia de nuestros objetos, eliminando la complejidad de los JOINs.

6.3. Operadores lógicos y de comparación: Gestión de predicados complejos

Un **predicado** es simplemente la condición que ponemos en el `WHERE` y que debe devolver `true` o `false`.

1. Operadores de Comparación Básicos

Funcionan igual que en Java o SQL. Se utilizan para comparar tipos primitivos (números, fechas, cadenas).

- **Relacionales:** `=`, `!=` (o `<>`), `<`, `>`, `<=`, `>=`.
- **Lógicos:** `AND`, `OR`, `NOT`.
- **Matemáticos:** `+`, `-`, `*`, `/`.

Ejemplo:
SQL
```
WHERE e.edad >= 18 AND e.salario < 30000
```

2. Comparación de Identidad (Objetos)

En SQL, para ver si una factura pertenece al cliente 5, comparas un número: `WHERE id_cliente = 5`. En OQL, comparamos instancias.

Podemos comparar si una referencia apunta a un objeto específico (comparación de OID implícita).

SQL
```
-- Suponiendo que 'unClienteVIP' es un objeto Java que pasamos como
parámetro (:param)
SELECT f
FROM Factura f
WHERE f.cliente = :unClienteVIP
```

Aquí no estamos comparando el DNI ni el nombre. Estamos preguntando: *"¿El objeto al que apunta `f.cliente` es físicamente el mismo objeto en memoria que `unClienteVIP`?"*.

3. El Reto de las Colecciones (`IN`, `CONTAINS`)

Este es el punto más complejo y potente. En Java, un objeto suele tener listas dentro.

- Clase `Departamento` tiene `List<Empleado> empleados`.

¿Cómo pregunto: **"Dame los departamentos que tengan AL MENOS un empleado que gane más de 50.000€"**?

En SQL harías un `JOIN`. En OQL, verificamos la colección directamente.

A. El operador `IN` (Variable iteradora)

Sirve para "entrar" en la colección y evaluar cada elemento.

SQL
```
SELECT d
FROM Departamento d, IN(d.empleados) e
WHERE e.salario > 50000
```

- **Explicación:** "Recorre los departamentos (`d`). Para cada uno, recorre su lista interna de empleados (`e`). Si encuentras un `e` que cumpla la condición, selecciona el departamento `d`".

B. El operador `CONTAINS` (Pertenencia directa)

Sirve para ver si un objeto concreto está en una lista.

SQL
```
WHERE d.empleados CONTAINS :empleadoJuan
```

C. El operador `IS EMPTY`

Verifica si la colección está vacía o no (equivalente a `List.isEmpty()` en Java).

SQL
```
SELECT d FROM Departamento d
WHERE d.empleados IS EMPTY
-- Devuelve departamentos sin gente (sin hacer count)
```

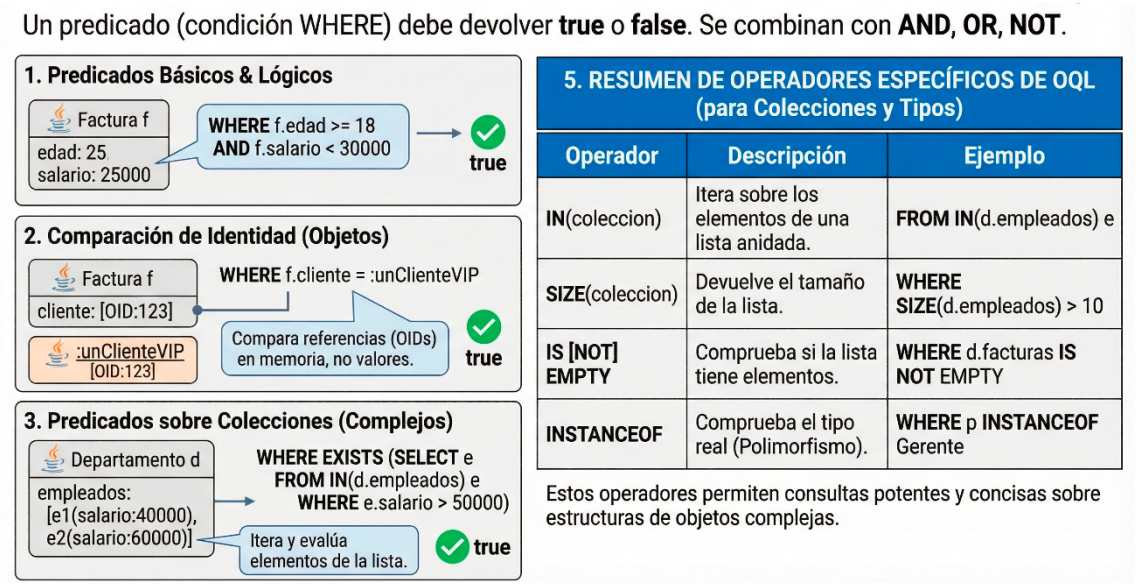
4. Coincidencia de Patrones (Strings)

Para buscar texto parcial, usamos `LIKE` con comodines, igual que en SQL.

- `%` (Porcentaje): Coincide con cualquier cadena de caracteres.
- `_` (Guion bajo): Coincide con un solo carácter.

SQL
```
-- Busca productos que empiecen por "Silla"
WHERE p.nombre LIKE "Silla%"
```

5. Resumen de Operadores Específicos de OQL



Ejemplo de Predicado Complejo

Vamos a combinarlo todo. **Objetivo:** Buscar "Proyectos que tengan más de 5 empleados, donde el jefe del proyecto se llame 'Ana' y que contengan al menos un empleado con salario > 3000".

SQL
```
SELECT p
FROM Proyecto p, IN(p.equipo) emp
WHERE SIZE(p.equipo) > 5          -- Condición de Colección
(Tamaño)
  AND p.jefe.nombre = 'Ana'       -- Navegación simple
  AND emp.salario > 3000          -- Condición sobre elemento de la
colección
```

Fíjate en la elegancia: hemos expresado una lógica que en SQL requeriría 3 tablas, claves foráneas, `GROUP BY`, `HAVING` y `JOINs`, en solo 4 líneas de código legible orientado a objetos.

6.4. Navegación por punteros: Uso del operador punto (.)

La **navegación** es el proceso de recorrer el grafo de objetos desde una entidad raíz hasta sus hijos, nietos o bisnietos, utilizando las referencias que ya existen en el modelo de clases.

El operador mágico es el **punto (.)**.

1. La Filosofía: "Camina, no reúnas"

Para entender la diferencia, imagina que quieres saber la **Capital del País de nacimiento del Autor de un Libro**.

- **Mentalidad SQL (Relacional):** Tienes 3 tablas desconectadas (`Libros`, `Autores`, `Paises`). Tienes que coger pegamento (Foreign Keys) y unirlas todas en una tabla gigante temporal para encontrar la fila correcta.
- **Mentalidad OQL (Objetos):** Tienes el objeto `Libro`. Ese libro ya "sabe" quién es su autor (tiene un puntero/OID). El autor "sabe" su país. Simplemente "caminas" de uno a otro.

2. Sintaxis de Travesía (Traversal)

La sintaxis es idéntica a acceder a atributos públicos en Java, pero aplicada dentro de la cadena de texto de la consulta.

Supongamos estas clases anidadas: `Pedido` -> tiene un `Cliente` -> tiene una `Direccion` -> tiene una `Ciudad` (String).

Consulta: *"Dame los pedidos enviados a clientes de París".*

SQL
```
SELECT p
FROM   Pedido p
WHERE  p.cliente.direccion.ciudad = 'París'
```

¿Qué ocurre internamente?

El motor de la base de datos realiza "Joins Implícitos".

1. Carga el OID del `cliente` dentro del `Pedido`.
2. Salta a ese objeto y busca el OID de `direccion`.
3. Salta a ese objeto y lee el String ciudad. Todo esto ocurre de forma transparente y optimizada usando índices físicos.

3. Reglas de Navegación

Aunque parece fácil, hay dos reglas de oro que dictan cuándo puedes usar el punto y cuándo no.

Regla A: Navegación a través de Referencias Simples (1:1 o N:1)

Si el atributo es un objeto único, **puedes** usar el punto para seguir bajando.

- `factura.cliente` (Correcto: un cliente es un objeto único).
- `factura.cliente.nombre` (Correcto).

Regla B: La Barrera de las Colecciones (1:N)

Si el atributo es una colección (`List`, `Set`, `Array`), **NO puedes** atravesarla directamente con un punto.

- `departamento.empleados` (Esto es una Lista de empleados).
- `departamento.empleados.nombre` -> **¡ERROR!**

¿Por qué falla? Porque `empleados` no tiene un atributo "nombre". empleados es una `List`. La lista tiene métodos como `size()` o `isEmpty()`, pero no tiene nombre. Quienes tienen nombre son los objetos dentro de la lista.

La Solución: Para atravesar una colección, necesitas usar una variable iteradora (como vimos en el punto 10.6.3 con `IN`).

SQL
```
-- MAL (Intento de cruzar la colección directamente)
SELECT d FROM Departamento d WHERE d.empleados.salario > 1000

-- BIEN (Usamos IN para entrar en la colección)
SELECT d FROM Departamento d, IN(d.empleados) e WHERE e.salario >
1000
```

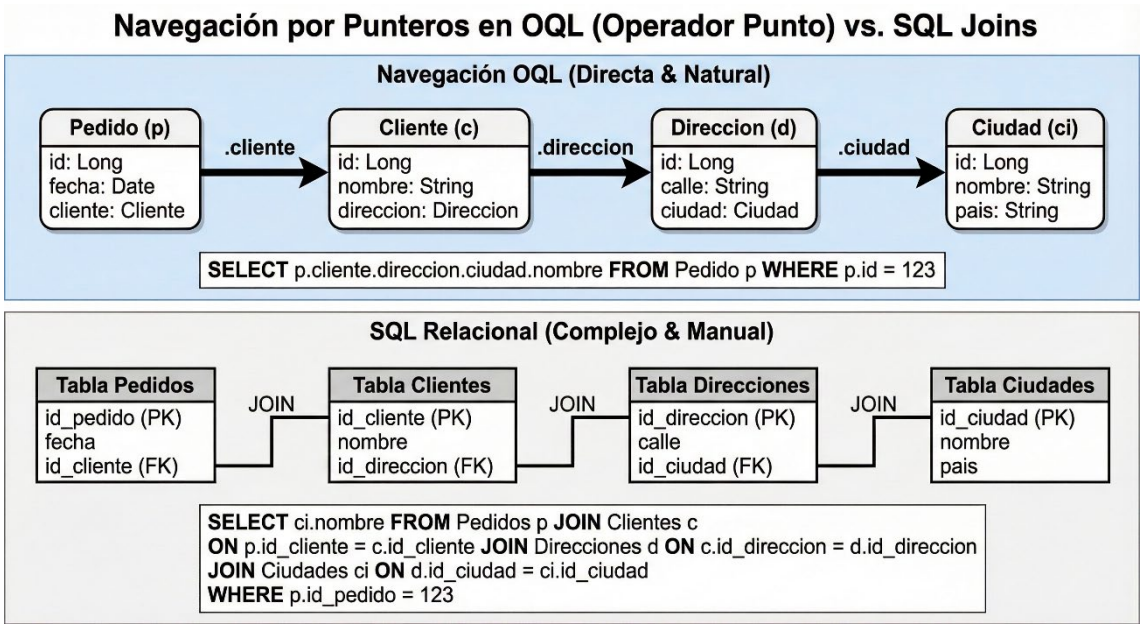
4. Seguridad ante Nulos (`NullPointer` en consultas)

En Java, si haces `coche.motor.cilindrada` y `motor` es `null`, el programa explota (`NullPointerException`).

En OQL pasa algo similar, pero más seguro:

- Si durante la navegación, uno de los pasos intermedios es `null` (ej: el cliente no tiene dirección), la expresión completa suele evaluarse como `false` o `null`, pero **no lanza error** que detenga la consulta (dependiendo de la implementación estricta del motor).
- Sin embargo, es buena práctica filtrar los nulos si la lógica lo requiere:

SQL
```
WHERE p.cliente.direccion IS NOT NULL
  AND p.cliente.direccion.ciudad = 'Madrid'
```



5. Resumen Visual

Tipo de Relación Java	Atributo	¿Puedo usar . para bajar más?	Ejemplo
Primitivo	`int edad`	NO (Es el final del camino)	`p.edad`
Objeto (1:1)	`Direccion dir`	SÍ	`p.dir.calle`
Colección (1:N)	`List<Pago> pagos`	**NO** (Debes usar `IN`)	`IN(p.pagos) pag`

7. Recuperación, modificación y borrado de información

7.1. Almacenamiento y persistencia: El método `store()` / `persist()`

En una Base de Datos Orientada a Objetos, persistir información significa tomar un objeto que está en la memoria RAM (volátil) y copiarlo al disco duro (permanente).

Dependiendo de la tecnología que uses, el método cambia de nombre, pero la filosofía es la misma:

- **db4o:** Usa el método **`db.store(objeto)`**.
- **JPA / ObjectDB:** Usa el método **`em.persist(objeto)`**.

1. El concepto: Persistencia por Alcanzabilidad (Persistence by Reachability)

Esta es la característica "mágica" del almacenamiento de objetos.

Imagina que tienes un objeto complejo: Un `Pedido` que contiene un `Cliente`, y ese pedido tiene una lista de 50 `LineasDeDetalle`, y cada línea tiene un `Producto`.

En SQL: Tendrías que escribir un script complejo:

1. Insertar Cliente (si no existe).
2. Insertar Pedido (obtener su ID generado).
3. Bucle for: Insertar cada Línea usando el ID del Pedido.

En BDOO: Solo tienes que guardar la raíz.

Java
```
// Java
db.store(miPedido);
```

La base de datos actúa como una araña:

1. Recibe el objeto `miPedido`.
2. Mira qué objetos están conectados a él (el Cliente, la lista de Líneas).
3. Mira qué objetos están conectados a esos (los Productos).
4. **Lo guarda TODO automáticamente.**

Si la base de datos detecta que el `Cliente` ya existía, simplemente guarda una referencia (puntero) a él. Si detecta que es nuevo, lo crea. Tú no tienes que preocuparte por el orden.

2. Ejemplo Práctico (db4o)

Vamos a guardar un equipo de Fórmula 1 con sus pilotos.

Java
```
// 1. Creamos los objetos en memoria RAM (Java puro)
Equipo ferrari = new Equipo("Ferrari");
Piloto p1 = new Piloto("Leclerc", 100);
Piloto p2 = new Piloto("Sainz", 95);

// 2. Conectamos los objetos (Asociación)
ferrari.addPiloto(p1);
ferrari.addPiloto(p2);

// 3. Abrimos la base de datos
ObjectContainer db = Db4oEmbedded.openFile("formula1.wap");

try {
    // 4. EL MOMENTO DE LA VERDAD
    // Solo guardamos al "padre". La DB guarda a los hijos por
cascada.
    db.store(ferrari);

    System.out.println("Equipo y pilotos guardados.");
} finally {
```

```
    db.close();
}
```

3. Diferencia entre Insertar y Actualizar

Una duda común es: *¿Cómo sabe la base de datos si estoy creando un objeto nuevo o actualizando uno viejo?*

El método `store()` (o `persist()`) es inteligente, pero funciona diferente según el motor:

A. Objetos Nuevos (Insert)

Si el objeto acaba de ser creado con `new` y nunca ha tocado la base de datos, no tiene OID (Identificador). Al hacer `store()`, la BDOO le asigna un OID y lo escribe en disco.

B. Objetos Existentes (Update)

Si recuperaste el objeto de la base de datos previamente (mediante una consulta), el objeto en memoria tiene un OID "pegado" internamente. Al hacer `store()` sobre él, la base de datos ve que ya tiene OID y sobrescribe los datos antiguos con los nuevos.

¡Cuidado en db4o! Si creas un objeto `new Piloto("Leclerc")` que es idéntico a uno que ya existe en la DB, y haces `store()`, **se creará un duplicado**. Para la BDOO, si no tiene OID, es nuevo. Para actualizar, **primero tienes que leer el objeto de la DB**, modificarlo y luego guardarlo.

4. JPA / ObjectDB: La sutileza de la Transacción

En el estándar JPA (usado por ObjectDB), el método `persist()` solo sirve para objetos nuevos. Para guardar cambios en objetos que ya existen, **no hace falta llamar a ningún método.**

Java
```
// Ejemplo JPA / ObjectDB
em.getTransaction().begin();

// 1. Recuperamos
Piloto p = em.find(Piloto.class, 1);

// 2. Modificamos en memoria
p.setPuntos(200);

// 3. Commit
// ¡Magia! Al hacer commit, JPA detecta que 'p' se ha modificado y
lo guarda solo.
// No hace falta llamar a em.store(p).
em.getTransaction().commit();
```

Esta característica ("Dirty Checking") hace que el código sea aún más limpio, ya que trata la base de datos como si fuera una colección en memoria.

7.2. Recuperación y refresco de objetos

Cuando trabajamos con SQL (JDBC), el proceso es "sin estado": lanzas una consulta, te dan datos copiados, cierras la conexión y te olvidas.

En una BDOO, el contenedor mantiene una relación "viva" con los objetos que tienes en memoria.

1. La Caché de Objetos (Identity Map)

Cuando recuperas un objeto de la base de datos (por ejemplo, `Cliente Juan`), el motor no solo te devuelve el objeto, sino que **se lo apunta en una caché interna** (llamada Mapa de Identidad).

¿Por qué hace esto? Para garantizar la identidad única. Si ejecutas dos consultas distintas y ambas devuelven a "Juan":

1. Consulta A: `Dame todos los Juanes`.
2. Consulta B: `Dame el cliente con saldo > 0`. La base de datos verifica su caché. Si "Juan" ya está cargado en la memoria RAM, **te devuelve la referencia al mismo objeto exacto** que ya tenías, no crea una copia nueva.

Esto es muy eficiente, pero genera un problema de sincronización: **La realidad en el disco y la realidad en tu memoria RAM pueden divergir.**

2. El problema de la Actualización (RAM -> Disco)

Supongamos que modificas un objeto en Java.

Java
```
cliente.setNombre("Nuevo Nombre");
// En este punto, la RAM dice "Nuevo Nombre", el Disco dice "Viejo
Nombre".
```

Para sincronizar hacia el disco, usamos `store()` (o `commit` en JPA). Sin embargo, en motores como **db4o**, existe una trampa mortal llamada **Profundidad de Actualización (Update Depth)**.

La Trampa

Imagina que modificas un objeto que está dentro de otro (anidado):

Java
```
coche.getMotor().setPotencia(200); // Modificamos el hijo
db.store(coche); // Guardamos al padre
```

- **Lo que esperas:** Que se guarde la nueva potencia.
- **Lo que pasa a menudo:** No se guarda nada.

¿Por qué? Por defecto, al hacer store(coche), la base de datos solo revisa los atributos directos del coche (color, marca). No entra a revisar si los objetos hijos (motor) han cambiado, para ahorrar rendimiento.

Solución:

1. **Guardar explícitamente al hijo:** `db.store(coche.getMotor())`.
2. **Configurar Cascada:** Activar `CascadeOnUpdate` como vimos en el punto 10.4.3.
3. **Persistencia Transparente:** Usar librerías que detectan cambios automáticamente (estándar en ObjectDB/JPA).

3. El Refresco de Objetos (Disco -> RAM)

Ahora el caso contrario. Imagina que has hecho cambios en un objeto en memoria, te arrepientes y quieres "**deshacer**" los cambios volviendo al estado que tiene guardado la base de datos. O imagina que estás en un sistema Cliente-Servidor y otro usuario ha cambiado el precio de un producto mientras tú lo tenías en pantalla. Tu objeto Java está obsoleto ("stale").

Para esto sirve el método **`refresh()`**.

Java
```
// Sintaxis db4o
// El segundo parámetro es la profundidad (cuántos niveles de hijos
refrescar)
db.ext().refresh(miProducto, 2);
```

Qué hace internamente:

1. La base de datos va al disco duro a buscar los datos actuales del objeto.
2. **Sobrescribe** brutalmente los valores de las variables de tu objeto en memoria Java.
3. Ahora tu RAM y el Disco están sincronizados de nuevo.

7.3. Eliminación de información: Borrado de instancias y gestión de referencias huérfanas

El borrado en BDOO difiere del SQL en un aspecto fundamental: **no hay restricciones de Clave Foránea (Foreign Key) activas por defecto** que te impidan borrar un padre si tiene hijos. El sistema confía en ti, y eso conlleva responsabilidad.

1. El método de borrado (`delete`)

Para eliminar un objeto, primero debes tenerlo "a mano" (cargado en memoria o recuperado por una consulta). No se suele hacer "borra donde ID=5" sin cargar el objeto.

- **En db4o:** `db.delete(objeto);`
- **En JPA/ObjectDB:** `em.remove(objeto);`

El proceso:

1. Buscas el objeto: `Piloto p = buscarPiloto("Alonso");`
2. Ordenas el borrado: `db.delete(p);`
3. El motor marca el OID de ese objeto como "libre" en el disco y el espacio queda disponible.

2. El problema de las Referencias Huérfanas

Imagina la siguiente situación:

1. Tienes un objeto `Coche` que apunta a un objeto `Motor`.
2. Haces `db.delete(motor)`.
3. **¡Peligro!** El objeto `Coche` **sigue teniendo el OID del motor** guardado en su atributo. La base de datos no va automáticamente al coche a poner ese campo a `null`.

Cuando intentes recuperar el `Coche` y acceder a su motor:

- El sistema intentará ir a la dirección de memoria del motor borrado.
- Dependiendo del motor de base de datos, obtendrás un `null`, una excepción o un objeto vacío.
- Esto se conoce como **Referencia Colgante (Dangling Reference)**.

Regla de Oro: Antes de borrar un objeto, es tu responsabilidad asegurarte de que nadie más lo está apuntando, o actualizar manualmente a los que lo apuntan para poner sus referencias a `null`.

3. Borrado en Cascada (Cascade Delete)

¿Y si quiero borrar un `Pedido` y que se borren automáticamente todas sus `LineasDeDetalle`? No quiero tener que borrar 50 líneas a mano una por una.

Para esto configuramos el **Borrado en Cascada**.

- **Configuración (db4o):**

Java
```
config.objectClass(Pedido.class).cascadeOnDelete(true);
```

- **Comportamiento:** Al ejecutar `db.delete(miPedido)`, el motor busca todos los objetos referenciados por él (las líneas) y los borra también.

¡Cuidado con la Cascada! Imagina que configuras cascada en Estudiante -> Curso. Si borras al estudiante "Juan", y la cascada está activa... **¡se borrará el Curso de Matemáticas de la base de datos!** Y todos los demás alumnos se quedarán sin curso.

- **Uso correcto:** Relaciones de composición fuerte (Padre-Hijo exclusivo, como Factura-Líneas).
- **Uso incorrecto:** Relaciones compartidas (Muchos a Muchos).

4. La "Basura" en la Base de Datos (Fuga de datos)

En Java, existe el **Garbage Collector (GC)**. Si dejas de usar un objeto en RAM, el GC lo detecta y libera la memoria.

En una BDOO, NO existe Garbage Collector automático por defecto.

Si tienes un `Coche` y le cambias el motor:

Java
```
Motor motorViejo = coche.getMotor();
Motor motorNuevo = new Motor("V8");
coche.setMotor(motorNuevo);
db.store(coche); // Guardamos el coche con el nuevo motor
```

¿Qué pasó con el `motorViejo`?

- En Java: El GC lo borra de la RAM.
- En la BDOO: **Se queda en el disco duro para siempre.** Ya nadie apunta a él, pero ocupa espacio. Es un objeto "zombie".

Para evitar que tu base de datos crezca infinitamente con datos inútiles, debes acordarte de borrar explícitamente los objetos que reemplazas si ya no sirven.

Resumen del CRUD en BDOO

Con esto cerramos el ciclo de manipulación de datos. Aquí tienes la "chuleta" rápida:

Operación	Método Clave (db4o)	Precaución Principal
Create / Update	`store(obj)`	Cuidado con la profundidad (hijos no guardados).
Read	`query(predicate)` / OQL	Cuidado con la profundidad de activación (`nulls`).
Delete	`delete(obj)`	Cuidado con dejar huérfanos o borrar de más (cascada).

7.4. Gestión de Transacciones: Garantizando ACID con commit() y rollback()

En una base de datos, una **Transacción** es un conjunto de operaciones que se consideran una única unidad indivisible de trabajo. La regla de oro es: **"O se hace todo, o no se hace nada"**.

Las BDOO (como ObjectDB o db4o) cumplen con las propiedades **ACID**:

- **A**tomicidad: Todo o nada.
- **C**onsistencia: La base de datos pasa de un estado válido a otro válido.
- **I**solation (Aislamiento): Lo que yo hago no lo ven otros hasta que termino.
- **D**urabilidad: Una vez guardado, persiste aunque se queme el servidor.

1. El modelo de transacción implícita

A diferencia de JDBC, donde a veces tienes que hacer `conn.setAutoCommit(false)`, en la mayoría de BDOO (db4o y ObjectDB en modo JPA) **la transacción está siempre activa**.

Desde el momento en que abres el contenedor (`ObjectContainer` o `EntityManager`), estás dentro de una transacción.

- Todo lo que haces (`store`, `delete`) ocurre en una "zona temporal" o búfer de transacciones.
- Nada se escribe permanentemente en el disco duro hasta que tú das la orden explícita.

2. Confirmar cambios: `commit()`

El método `commit()` es el botón de "Guardar" definitivo. Cuando lo invocas:

1. El motor coge todos los objetos nuevos, modificados y borrados que están en el búfer.
2. Los escribe físicamente en el archivo de la base de datos de forma segura.
3. Limpia el búfer para empezar una nueva transacción limpia.

Java
```
// Hacemos cambios
Piloto p = new Piloto("Fernando");
db.store(p);

// ¡OJO! Aquí "p" todavía NO está seguro en el disco.
// Si se va la luz ahora, se pierde el piloto.

db.commit(); // Ahora SÍ está guardado permanentemente.
```

Más info: Al cerrar la base de datos con `db.close()`, la mayoría de los motores hacen un `commit()` automático implícito, pero es mala práctica confiar en ello. Haz commit explícito cuando termines una unidad lógica de trabajo.

3. Deshacer cambios: `rollback()`

El método `rollback()` es el botón de "Deshacer" (Ctrl+Z). Sirve para descartar todo lo que hayas hecho desde el último commit.

Es vital para el manejo de errores. Si estás haciendo una operación compleja y falla el paso 5 de 10, no quieres que los pasos 1-4 se queden guardados a medias. Haces rollback y la base de datos vuelve al estado exacto que tenía al principio.

Efecto secundario en Objetos Java: El `rollback()` no solo limpia la base de datos, sino que debe limpiar tu memoria RAM.

- El motor de base de datos refresca (Refresh) automáticamente los objetos en memoria para que vuelvan a tener los valores antiguos.
- Si habías creado objetos nuevos, estos quedan "desconectados" de la base de datos.

4. Ejemplo Clásico: La Transferencia Bancaria

No hay mejor ejemplo para entender esto que mover dinero de A a B. Si restamos dinero de A y el programa falla antes de sumarlo a B, el dinero desaparece.

Java

```java
public void transferir(Cuenta origen, Cuenta destino, double
cantidad) {
    try {
        // 1. Validaciones
        if (origen.getSaldo() < cantidad) throw new Exception("Sin
fondos");

        // 2. Operaciones en memoria (en el búfer de transacción)
        origen.retirar(cantidad);
        db.store(origen); // "Guardado" provisional

        // SIMULACIÓN DE ERROR: Imagina que aquí se va la luz o
falla el código
        // if (true) throw new RuntimeException("Error inesperado
en la red");

        destino.ingresar(cantidad);
        db.store(destino); // "Guardado" provisional

        // 3. El momento de la verdad
        db.commit();
        System.out.println("Transferencia exitosa.");

    } catch (Exception e) {
        // 4. Si algo falla, REBOBINAMOS todo
        System.out.println("Error en la transferencia: " +
e.getMessage());

        db.rollback();

        // Ahora 'origen' vuelve a tener su saldo original en la DB
        // y db4o restaura el valor en el objeto Java
automáticamente.
    }
}
```

5. Concurrencia: Cuando varios usuarios tocan lo mismo

En un entorno multiusuario (Cliente-Servidor), las transacciones también gestionan el **bloqueo (Locking)**.

- **Lectura:** Varios usuarios pueden leer el mismo objeto a la vez.
- **Escritura:**
    - **Optimista (Por defecto en BDOO):** El sistema asume que no habrá conflictos. Si dos usuarios modifican el mismo objeto y hacen commit, el último gana (o salta una excepción de colisión de versiones, dependiendo de la config).
    - **Pesimista:** Bloqueas el objeto para ti. Nadie más puede leerlo ni escribirlo hasta que hagas commit. (Menos común en BDOO puras por ser menos eficiente).

8. Tipos de datos objeto: atributos y métodos

8.1. Persistencia de atributos: Tipos simples, objetos anidados y tipos transient

La regla general de las Bases de Datos Orientadas a Objetos es la **Persistencia Transparente**. Esto significa que el motor intenta guardar el estado del objeto tal cual está en la memoria RAM, sin que tú tengas que convertir tipos de datos manualmente (como harías en JDBC pasando de `java.util.Date` a `java.sql.Date`).

Sin embargo, hay reglas específicas para cada categoría de atributo.

1. Tipos Simples (Primitivos y Wrappers)

Son los ladrillos básicos. Todos los tipos estándar de Java se guardan de forma directa y nativa.

- **Primitivos:** `int`, `boolean`, `double`, `char`, `long`, `byte`.
- **Envoltorios (Wrappers):** `Integer`, `Boolean`, `Double`, etc.
- **Cadenas:** `String`.
- **Fechas:** `java.util.Date`, `Calendar`, `java.time.LocalDate` (dependiendo de la versión del motor).
- **Big Numbers:** `BigDecimal`, `BigInteger` (vitales para aplicaciones financieras).

Más info: **`null`**, A diferencia de los tipos primitivos (`int` siempre es 0 por defecto), los Wrappers y Strings pueden ser `null`. La BDOO guarda perfectamente este valor `null`.

2. Objetos Anidados y Colecciones (Relaciones)

Aquí reside la potencia de la BDOO. Cuando un atributo no es un tipo simple, sino otra clase o una colección, el motor no guarda una copia plana, sino que mantiene la estructura.

- **Referencias (1:1):** Si tienes `private Direccion direccion`, la BDOO guarda el objeto `Direccion` y mantiene el puntero.
- **Arrays:** `String[]`, `int[]`, `Persona[]`. Se guardan como bloques contiguos.
- **Colecciones (1:N):** Soporte nativo para `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `HashMap`, etc.
    - La base de datos guarda la colección y preserva el orden (si es una List) o la unicidad (si es un Set).

3. Atributos `transient`: Lo que NO queremos guardar

A veces, una clase tiene atributos que **no deben ir al disco duro**. Son datos temporales, calculados o sensibles que solo tienen sentido en la memoria RAM mientras el programa corre.

Para evitar que la base de datos los guarde, usamos la palabra reservada de Java: **`transient`**.

Casos de uso típicos para `transient`:

1. **Datos Calculados:** Si guardas `fechaNacimiento`, no hace falta guardar `edad`. La edad se calcula en el momento. Si la guardas, al año siguiente el dato en la base de datos será incorrecto.

2. **Recursos del Sistema:** Un objeto `FileInputStream`, un `Thread` o un `Socket`. No puedes guardar una "conexión abierta a internet" en el disco duro.
3. **Cachés temporales:** Variables auxiliares usadas para acelerar cálculos momentáneos.
4. **Seguridad:** Una contraseña en texto plano que solo usas para verificar y luego descartas.

Ejemplo:
Java
```java
public class Cliente {
    // SE GUARDA: Dato esencial
    private String nombre;
    private Date fechaNacimiento;

    // NO SE GUARDA: Se calcula a partir de fechaNacimiento
    private transient int edad;

    // NO SE GUARDA: Es un recurso del sistema operativo
    private transient Scanner lectorConsola;

    public int getEdad() {
        // Al recuperar el objeto de la DB, 'edad' vendrá como 0.
        // Debemos recalcularla.
        return calcularEdad(this.fechaNacimiento);
    }
}
```

Importante al recuperar: Cuando la BDOO recupera un objeto `Cliente` del disco, el atributo `edad` (transient) vendrá con su **valor por defecto de Java** (`0` para int, `null` para objetos). Es tu responsabilidad volver a calcularlo o inicializarlo.

4. Atributos `static` (Variables de Clase)

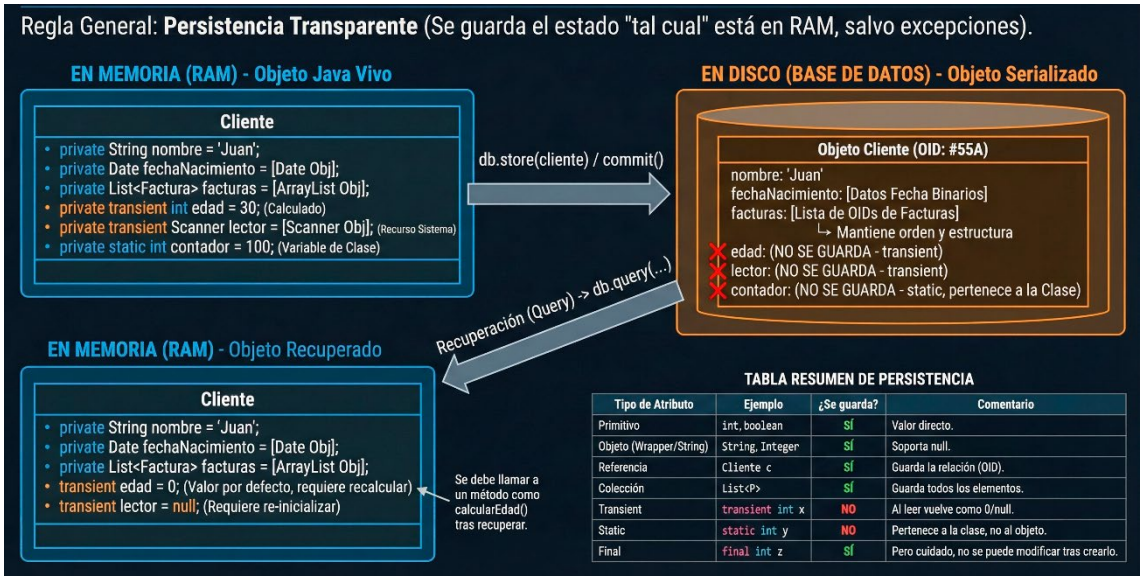
Las variables estáticas (`private static int contador`) pertenecen a la **Clase**, no a los objetos individuales.

- Las BDOO están diseñadas para guardar **instancias** de objetos.
- Por norma general, **los atributos estáticos NO se guardan**.
- Si cambias un valor estático en tiempo de ejecución y cierras el programa, ese valor se pierde, aunque hagas `commit()`.

5. Tabla Resumen de Persistencia

Tipo de Atributo	Ejemplo	¿Se guarda?	Comentario
Primitivo	`int`, `boolean`	**SÍ**	Valor directo.
Objeto (Wrapper/String)	`String`, `Integer`	**SÍ**	Soporta `null`.
Referencia	`Cliente c`	**SÍ**	Guarda la relación (OID).

Tipo de Atributo	Ejemplo	¿Se guarda?	Comentario
Colección	`List<P>`	**SÍ**	Guarda todos los elementos.
Transient	`transient int x`	**NO**	Al leer vuelve como 0/null.
Static	`static int y`	**NO**	Pertenece a la clase, no al objeto.
Final	`final int z`	**SÍ**	Pero cuidado, no se puede modificar tras crearlo.



8.2. Almacenamiento de comportamiento: ¿Cómo gestiona la base de datos los métodos?

A diferencia de las bases de datos relacionales, que tienen "Procedimientos Almacenados" (Stored Procedures) escritos en PL/SQL dentro del servidor, las BDOO delegan la lógica completamente en las clases Java.

1. La separación: Estado vs. Comportamiento

Para entender cómo funciona, hay que visualizar dónde vive cada cosa:

1. **El Archivo de Base de Datos (`.odb` / `.wap`):** Aquí se guardan los valores de los atributos (**El Estado**). Se guarda que `p1` tiene `puntos = 100`.
2. **El Classpath de Java (`.jar` / `.class`):** Aquí se guardan las instrucciones de los métodos (**El Comportamiento**). Aquí está el código que dice `calcularBonus() { return puntos * 10; }`.

Cuando recuperas un objeto de la base de datos:

1. El motor lee los datos del disco.
2. El motor busca en tu proyecto Java la clase correspondiente (usando *Reflection*).
3. Crea una instancia vacía de esa clase.
4. Rellena los atributos con los datos del disco.
5. **Resultado:** Tienes un objeto vivo que tiene los datos antiguos pero que ejecuta el código que tienes actualmente en tu programa.

2. Ejecución de Métodos en Consultas

Una de las características más potentes de las BDOO es que permiten usar métodos dentro de las condiciones de búsqueda (`WHERE`), algo imposible en SQL estándar.

Ejemplo: Imagina que la clase `Factura` tiene un método complejo:

Java
```
public boolean estaVencida() {
    return fecha.before(new Date()) && !pagada;
}
```

Consulta en BDOO:
Java
```
// Podemos llamar al método directamente en la consulta
List<Factura> vencidas = db.query(new Predicate<Factura>() {
    public boolean match(Factura f) {
        return f.estaVencida(); // ¡Ejecución de lógica Java!
    }
});
```

¿Cómo funciona esto? El motor de la base de datos tiene que instanciar el objeto en la memoria (RAM), rellenar sus datos y luego ejecutar tu método Java `estaVencida()`.

- **Ventaja:** Reutilizas tu lógica de negocio. No tienes que duplicar la lógica de "vencimiento" en un SQL `WHERE date < NOW() AND paid = 0`.
- **Desventaja:** Si tienes 1 millón de facturas, instanciar 1 millón de objetos para ejecutar el método es lento. (Por eso se prefieren consultas sobre atributos directos siempre que sea posible).

3. Evolución del Comportamiento (Versioning)

Como el código no está guardado en la base de datos, tienes una flexibilidad enorme para cambiar la lógica sin tocar los datos.

Escenario:

- **Hoy:** El método `getSalarioNeto()` resta un 15% de impuestos. Guardas 1000 empleados.
- **Mañana:** Cambia la ley. Modificas tu código Java para que `getSalarioNeto()` reste un 20%.
- **Resultado:** No tienes que hacer ningún `ALTER TABLE` ni actualizar datos. Simplemente despliegas tu nuevo `.jar`. Cuando recuperes los empleados viejos, al llamar al método, se ejecutará la nueva lógica del 20% automáticamente sobre los datos existentes.

4. Polimorfismo y Clases Abstractas

La base de datos gestiona perfectamente el comportamiento polimórfico.

Si tienes una clase abstracta `Animal` y dos hijas `Perro` y `Gato`, y una lista `List<Animal>`:

- La base de datos guarda qué clase concreta es cada objeto.
- Al recuperar la lista y recorrerla:

Java
```
for (Animal a : zoologico) {
    a.hacerRuido();
}
```

- Si el objeto recuperado era un `Perro`, Java ejecutará el `hacerRuido()` de la clase `Perro` (guau).
- Si era un `Gato`, ejecutará el de `Gato` (miau).

Esto ocurre de forma transparente. La BDOO sabe instanciar la clase correcta para que el comportamiento sea el esperado.

8.3. Profundidad de activación (Activation Depth)

En una base de datos relacional (SQL), cuando haces un `SELECT * FROM Clientes`, obtienes los datos de esa tabla y punto. Si quieres sus facturas, tienes que hacer otra consulta o un JOIN explícito.

En una BDOO, los objetos forman una **red gigante** (un grafo).

- El `Cliente` apunta a `Facturas`.
- Las `Facturas` apuntan a `Productos`.
- Los `Productos` apuntan a `Fabricantes`.
- Los `Fabricantes` apuntan a `Paises`.

Si la base de datos fuera "codiciosa" y cargara todas las relaciones automáticamente al pedir el Cliente, acabaría cargando **la base de datos entera en la memoria RAM**.

Para evitar esto, las BDOO utilizan el concepto de **Profundidad de Activación**.

1. ¿Qué es la Profundidad de Activación?

Es el número de "saltos" o niveles de referencias que el motor de la base de datos recorre y carga en memoria automáticamente cuando recuperas un objeto raíz.

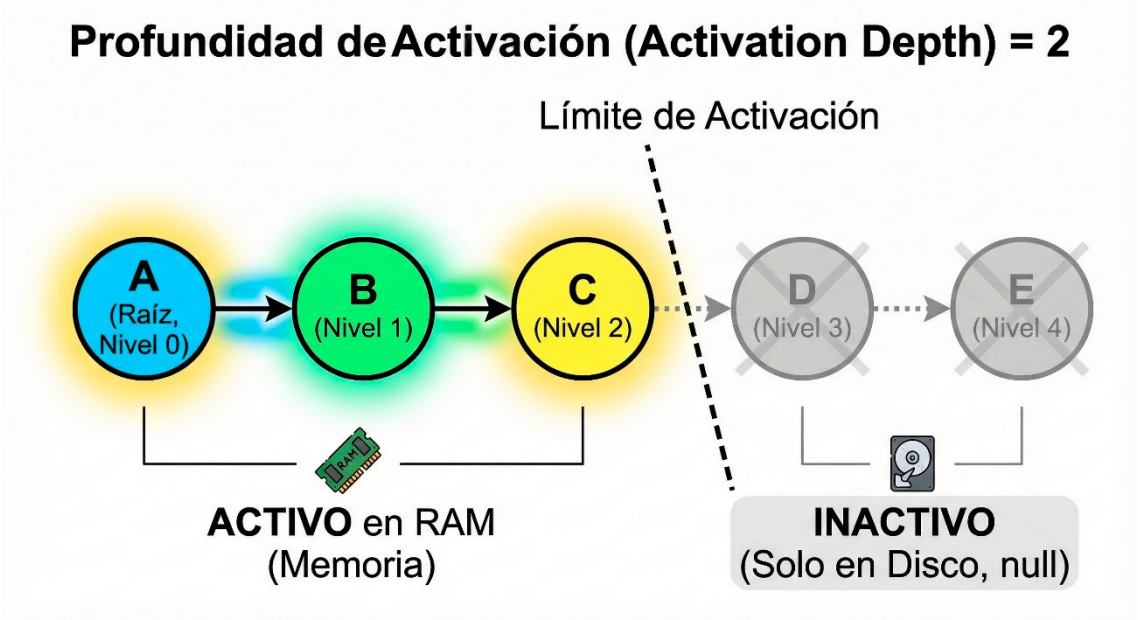
Imagina una cadena de objetos: `A` -> `B` -> C -> `D` -> `E`

Si la profundidad está configurada a **2**:

1. Recuperas `A`.

2. El motor carga `A` (Nivel 0).
3. El motor ve que A apunta a `B`, así que carga `B` (Nivel 1).
4. El motor ve que B apunta a `C`, así que carga `C` (Nivel 2).
5. **STOP.** El motor se detiene aquí.

¿Qué pasa con D? El objeto `C` en tu memoria Java tendrá una referencia a `D`, pero esa referencia estará **desactivada** (generalmente `null` o un proxy vacío). El objeto `D` existe en el disco duro, pero no en tu RAM.



2. El problema del "Falso Null"

El error más común de los principiantes en BDOO es creer que han perdido datos.

Java
```
// Configuramos profundidad baja (ej: 1)
Cliente c = db.query(...).get(0);

// Esto funciona (Nivel 1)
System.out.println(c.getDireccion().getCalle());

// Esto falla (Nivel 2 - No alcanzado)
// Lanza NullPointerException, aunque el dato existe en la DB
System.out.println(c.getDireccion().getCiudad().getNombre());
```

Si te encuentras un `NullPointerException` al navegar profundamente por tus objetos, el 99% de las veces no es que el dato falte, es que la **profundidad de activación** no ha llegado hasta allí.

3. Configuración de la Profundidad

Dependiendo del motor (especialmente en **db4o**), tienes tres formas de controlar esto:

A. Configuración Global (La fuerza bruta)

Puedes decirle a la base de datos: "Por defecto, carga siempre 10 niveles".

- **Ventaja:** Te olvidas de los nulos.
- **Desventaja:** El rendimiento cae en picado. Cargas miles de objetos innecesarios.

Java
```
// Configuración db4o
config.common().activationDepth(10);
```

B. Configuración Específica por Clase (La inteligente)

Puedes decir: "Para casi todo usa profundidad 5, pero para la clase `ArbolGenealogico`, quiero profundidad 20 porque necesito recorrerla entera".

Java
```
config.common().objectClass(ArbolGenealogico.class).minimumActivati
onDepth(20);
```

C. Activación Manual (Bajo demanda)

Esta es la estrategia más eficiente. Dejas la profundidad baja por defecto, y si en un momento dado necesitas bajar más, se lo pides explícitamente al contenedor.

Java
```
// Recuperamos el cliente (profundidad estándar)
Cliente c = ...;

// Queremos acceder a datos muy profundos que ahora mismo son null.
// Le decimos a la DB: "Activa este objeto específico 5 niveles
más".
db.activate(c, 5);

// Ahora ya no es null
System.out.println(c.getDireccion().getCiudad().getNombre());
```

4. Transparencia vs. Control

En motores modernos como **ObjectDB (JPA)**, este concepto se maneja mediante **Lazy Loading** (Carga Perezosa).

- No hay un número fijo de profundidad (5, 10...).
- Cuando tocas un atributo (`getLista()`), la base de datos intercepta la llamada y carga los datos en ese preciso instante (si la sesión sigue abierta).
- Es más transparente para el programador, pero requiere mantener la conexión abierta.

En motores "puros" o embebidos clásicos (**db4o**), la Profundidad de Activación es una configuración estática y dura que debes conocer para no volverte loco depurando.

9. Tipos de datos colección

9.1. Persistencia de Arrays y Listas: Manejo de ArrayList, LinkedList y arrays nativos

En Java, raramente trabajamos con objetos aislados. Lo normal es tener un `Pedido` que tiene muchas `Lineas`, o una `Clase` que tiene muchos `Alumnos`. Son relaciones **1:N** (Uno a Muchos).

Las BDOO soportan estas estructuras de forma **nativa**. No las "simulan" como hace SQL; las guardan tal cual existen en la memoria.

1. Arrays Nativos (`[]`)

Los arrays de Java (tamaño fijo) son ciudadanos de primera clase en las BDOO.

- **Arrays de Primitivos:** `int[]`, `byte[]`. Se guardan de forma extremadamente eficiente (como un bloque binario contiguo). Son ideales para guardar datos de señales, imágenes o matrices matemáticas.
- **Arrays de Objetos:** `String[]`, `Persona[]`. Se guardan como una secuencia de referencias (OIDs).

Ejemplo:
Java
```java
class Sensor {
    // Se guarda como un bloque binario comprimido en disco
    int[] lecturas = new int[1000];

    // Se guarda una lista de punteros a Strings
    String[] etiquetas = {"Temperatura", "Humedad"};
}
```

2. La Interfaz List (`ArrayList`, `LinkedList`)

Aquí está la verdadera potencia. Las BDOO permiten guardar directamente implementaciones de `java.util.List`.

Lo más importante es que **respetan el orden**.

- Si en tu `ArrayList` insertaste a "Juan" primero y a "Ana" después, cuando recuperes la lista de la base de datos dentro de un año, "Juan" seguirá estando en el índice 0.
- En SQL, el orden de las filas no está garantizado a menos que crees una columna artificial `orden_id` y hagas `ORDER BY`. En BDOO es implícito.

¿Qué pasa internamente?

- **`ArrayList`:** La base de datos guarda un array dinámico de OIDs (Identificadores de Objeto). Es muy rápido de leer.
- **`LinkedList`:** La base de datos guarda la estructura enlazada. Es menos común usarla para persistencia porque la lectura aleatoria es más lenta, pero está soportada.

3. El Adiós a la Tabla Intermedia

Para entender la ventaja, comparemos cómo se guarda un `Equipo` con muchos `Jugadores`.

- **En SQL (El dolor):**

  1. Creas tabla `EQUIPO`.
  2. Creas tabla `JUGADOR`.
  3. En `JUGADOR` pones una columna `FK_EQUIPO_ID`.
  4. Al recuperar, haces `SELECT * FROM JUGADOR WHERE FK_EQUIPO_ID = ?`.
  5. Reconstruyes la lista en Java manualmente.

- **En BDOO (La naturalidad):**

Java
```
public class Equipo {
    private String nombre;
    // La BDOO guarda esto directamente. Fin.
    private List<Jugador> plantilla = new ArrayList<>();
}
```

Al hacer `db.store(equipo)`, la base de datos recorre la lista, guarda a todos los jugadores (si no existían) y guarda la relación de pertenencia.

4. Precaución: El problema de las "Listas Gigantes"

Aunque es muy cómodo, hay un riesgo de rendimiento. Si tienes una clase `Pais` y le pones un atributo `List<Habitante> habitantes` con 40 millones de objetos:

- Cada vez que cargues el objeto `Pais`, la base de datos intentará instanciar la lista.
- Aunque use "Carga Perezosa" (Lazy Loading), gestionar una lista de 40 millones de referencias en un solo objeto es ineficiente.
- **Consejo:** Las colecciones directas son geniales para relaciones 1:N moderadas (ej: Factura -> Líneas). Para relaciones masivas, es mejor usar consultas (`Query`) en lugar de listas directas.

9.2. Conjuntos y Mapas: Uso de HashSet y HashMap

Las BDOO no solo guardan datos, guardan **estructuras de datos**. Esto significa que las reglas lógicas de las colecciones de Java se mantienen intactas cuando apagas el ordenador.

1. Conjuntos: `Set` / `HashSet`

Un `Set` es una colección que **no admite duplicados**. En el modelo relacional (SQL), para evitar duplicados en una relación 1:N, dependes de crear restricciones `UNIQUE` o claves primarias compuestas en tablas intermedias.

En una BDOO, simplemente usas un `HashSet`.

Funcionamiento en BDOO

Si tienes un objeto `Proyecto` con un conjunto de `Empleados`:

Java
```
public class Proyecto {
    private String nombre;
    // HashSet garantiza que un empleado no esté dos veces en el
mismo proyecto
    private Set<Empleado> equipo = new HashSet<>();

    public void agregar(Empleado e) {
        this.equipo.add(e);
    }
}
```

- **Persistencia:** Al guardar el `Proyecto`, la base de datos guarda la colección.
- **Regla de Unicidad:** Si intentas añadir el mismo objeto `Empleado` (mismo OID) dos veces, el `HashSet` lo ignorará silenciosamente, tal como hace en Java. Al recuperar los datos del disco, la colección vendrá limpia de duplicados.

¡Advertencia Vital (hashCode/equals)! Para que un `HashSet` funcione correctamente al recuperarse del disco, los objetos que guardas dentro deben tener implementados correctamente los métodos `hashCode()` y `equals()`.

- Si no lo haces, Java usa la dirección de memoria por defecto.
- Al cerrar y abrir el programa, la dirección de memoria cambia. El `HashSet` podría "perder" objetos o permitir duplicados lógicos porque sus códigos hash han cambiado.

2. Mapas: `Map` / `HashMap`

Los mapas (diccionarios o arrays asociativos) son estructuras Clave-Valor. Son increíblemente útiles, pero **SQL las odia**.

Para guardar un `HashMap<String, Integer>` en SQL, necesitas crear una tabla extra `TABLA_MAPA` con columnas `CLAVE`, `VALOR` y `FK_PADRE`. Es mucho trabajo para guardar una configuración simple.

En BDOO, los mapas son ciudadanos de primera clase.

Escenario de uso

Imagina una clase `Configuracion` que guarda parámetros variables.

Java
```
public class Configuracion {
    private String usuario;

    // Un mapa flexible: "resolucion" -> "1080p", "volumen" -> "80"
    private Map<String, String> preferencias = new HashMap<>();

    public void setPref(String clave, String valor) {
        preferencias.put(clave, valor);
    }
```

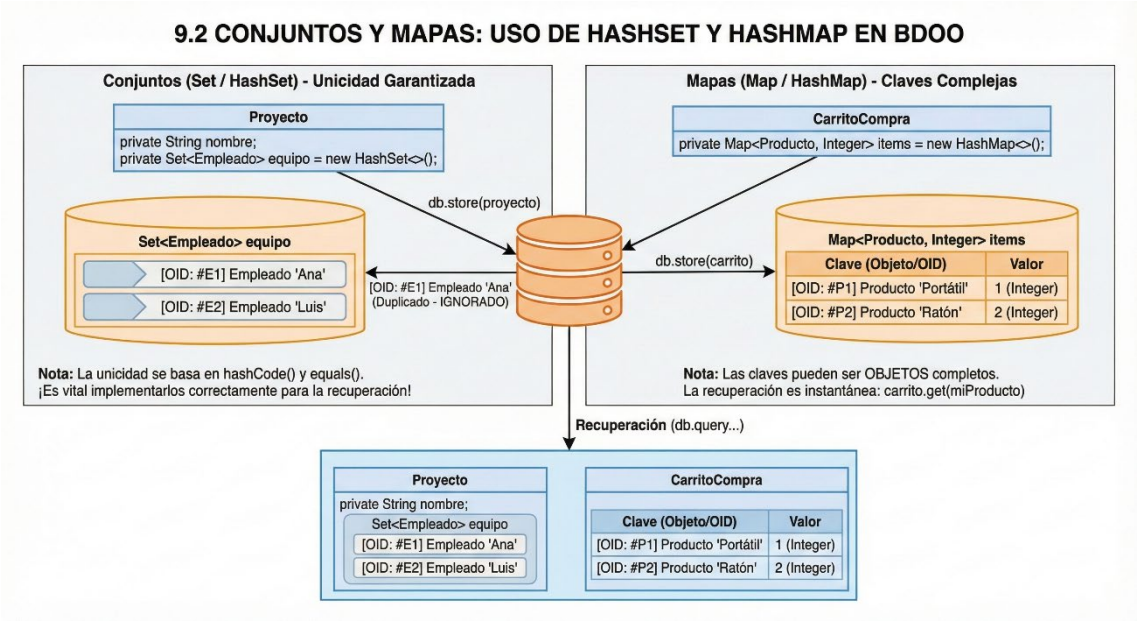
```
}
```

Persistencia de Claves Complejas

Lo impresionante es que las claves y los valores no tienen por qué ser Strings. Pueden ser **Objetos**.

Puedes tener un `Map<Producto, Integer>` para representar un carrito de la compra (Producto -> Cantidad).

- **En SQL:** Necesitas una tabla intermedia `LINEAS_CARRITO` con `FK_PRODUCTO` y columna `CANTIDAD`.
- **En BDOO:**



Java
```
// Mapa: Objeto Producto -> Cantidad (Integer)
private Map<Producto, Integer> carrito = new HashMap<>();
```

La base de datos guarda los OIDs de los productos como claves y los enteros como valores. Al recuperarlo, `carrito.get(miProducto)` funciona instantáneamente.

9.3. Gestión de relaciones: Implementación de cardinalidades 1:N y N:M

La **cardinalidad** define cuántos objetos de un lado se relacionan con cuántos del otro. Las BDOO resuelven esto usando **Referencias** (para el lado "1") y **Colecciones** (para el lado "N" o "M").

1. Relación Uno a Muchos (1:N)

Es la más común. Ejemplo clásico: **Un Departamento tiene muchos Empleados**.

- **En SQL:** No tocas la tabla `Departamento`. Vas a la tabla `Empleado` y añades una columna `ID_DEPARTAMENTO` (Clave Foránea).

- **En BDOO:** Es mucho más intuitivo. El departamento tiene una lista de sus empleados.

Implementación Java

Java
```
public class Departamento {
    private String nombre;

    // El lado "N": Una colección
    private List<Empleado> empleados = new ArrayList<>();
}

public class Empleado {
    private String nombre;

    // El lado "1": Una referencia simple (opcional, si quieres
bidireccionalidad)
    private Departamento trabajaEn;
}
```

Ventaja: Para saber quién trabaja en "Ventas", no haces una query buscando en toda la tabla de empleados. Simplemente haces `ventas.getEmpleados()`. Es navegación directa en memoria.

2. Relación Muchos a Muchos (N:M)

Esta es la "bestia negra" de SQL. Ejemplo: **Un Estudiante se matricula en muchos Cursos, y un Curso tiene muchos Estudiantes**.

- **En SQL:** Es obligatorio crear una tercera tabla artificial (`MATRICULAS`) que solo contiene IDs (`ID_ESTUDIANTE`, `ID_CURSO`).
- **En BDOO: No existen las tablas intermedias.** La relación se modela con dos colecciones cruzadas.

Implementación Java

Java
```
public class Estudiante {
    private String nombre;
    // Lado N
    private List<Curso> cursosMatriculados = new ArrayList<>();
}

public class Curso {
    private String titulo;
    // Lado M
    private List<Estudiante> inscritos = new ArrayList<>();
}
```

Al guardar los objetos, la base de datos se encarga de gestionar este cruce de punteros.

- Si borras un estudiante, la BDOO (dependiendo de la config) lo elimina de la lista `inscritos` de los cursos correspondientes para no dejar referencias rotas.

3. El reto de la Integridad Referencial (Métodos Helper)

Aquí hay una trampa importante. En SQL, si insertas una fila en la tabla intermedia, la relación queda establecida para ambos lados.

En Java, si haces esto:

Java
```
cursoJava.getInscritos().add(pepe);
// ¡ERROR DE LÓGICA!
```

...has añadido a Pepe en el curso, **PERO** no has añadido el curso en la lista de Pepe (`pepe.getCursosMatriculados()`). Tu modelo en memoria es inconsistente.

Para evitar esto, en BDOO es fundamental encapsular la gestión de relaciones en **Métodos de Conveniencia (Helper Methods)** que actualicen ambos lados a la vez.

Forma Correcta:
Java
```
// Dentro de la clase Curso
public void matricular(Estudiante e) {
    // 1. Añado al estudiante a mi lista
    this.inscritos.add(e);

    // 2. Me añado a mí mismo a la lista del estudiante
    e.getCursosMatriculados().add(this);
}
```

Ahora, al llamar a `cursoJava.matricular(pepe)`, la relación queda perfectamente sellada en ambos sentidos y lista para persistirse.

4. Unidireccional vs. Bidireccional

En SQL, las relaciones siempre son bidireccionales (si tienes la FK, puedes hacer JOIN desde cualquier lado).

En BDOO, tú decides para ahorrar memoria:

1. **Unidireccional:** ¿Realmente necesitas saber en qué cursos está un alumno? Si tu aplicación solo lista alumnos por curso, quizás la clase `Estudiante` no necesite la lista de cursos. Ahorras espacio y complejidad.
2. **Bidireccional:** Necesitas navegar en ambos sentidos. Requiere el doble de gestión (como vimos en el punto 3), pero ofrece máxima flexibilidad de consulta.

Resumen

1. El Concepto Fundamental

Las BDOO nacen para solucionar la **"Desadaptación de Impedancia"** (Impedance Mismatch).

- **El Problema:** Java usa objetos, herencia y grafos. SQL usa tablas planas y claves primarias. Traducir de uno a otro es costoso y complejo (requiere mucho código de mapeo).
- **La Solución:** La BDOO guarda el objeto tal cual está en memoria. **Java es el modelo de datos.**

2. Características Clave (Vs. SQL)

- **Identidad (OID):** Cada objeto tiene un identificador interno único e inmutable. No necesitamos crear claves primarias artificiales (`ID_CLIENTE`) en nuestras clases.
- **Herencia y Polimorfismo:** Soportados nativamente. Podemos guardar una lista mixta de `Perro` y `Gato` (hijos de `Animal`) y recuperarla sin trucos de tablas.
- **Objetos Complejos:** Se guardan estructuras anidadas (listas dentro de listas) y multimedia de forma eficiente sin fragmentar la información en múltiples tablas.

3. Motores y Arquitectura

- **Motores:** Hemos hablado de **db4o** (educativo, consultas nativas) y **ObjectDB** (profesional, estándar JPA).
- **Instalación:** Son bases de datos **embebidas**. Se instalan añadiendo una librería (`.jar`) al proyecto. No requieren instalar un servidor externo complejo.
- **Modos de uso:**
    - **Embebido:** La DB vive dentro de tu aplicación (máxima velocidad, monousuario).
    - **Cliente-Servidor:** Varios usuarios conectan por red a un servidor central.

4. Operaciones CRUD (Ciclo de Vida)

La gestión es transparente mediante el `ObjectContainer`:

- **Guardar (`store/persist`):** Persistencia por alcanzabilidad. Guardas al objeto "padre" y la base de datos guarda automáticamente a todos sus "hijos" y referencias.
- **Actualizar:** Se modifican los atributos en Java y se vuelve a guardar.
- **Borrar (`delete`):** Elimina el objeto. Hay que tener cuidado con dejar referencias huérfanas (punteros a la nada).
- **Transacciones:** Siguen el modelo ACID. Todo ocurre en memoria hasta que haces `commit()` (guardar en disco) o `rollback()` (deshacer).

5. Mecanismos de Consulta

Las BDOO ofrecen varias formas de recuperar datos sin usar SQL tradicional:

1. **QBE (Query By Example):** Creas un objeto "plantilla" (`new Piloto(null, 100)`). Fácil pero limitado (solo igualdad).
2. **Native Queries (Consultas Nativas):** Usas código Java (`if`, `return`) para filtrar. Seguro (Type-Safe) y potente.
3. **SODA:** Grafos de consulta de bajo nivel para búsquedas dinámicas.

4. **OQL (Object Query Language):** Estándar parecido a SQL (`SELECT p FROM Pais p`). Permite navegar por relaciones usando puntos (`p.ciudad.nombre`) en lugar de `JOINs`.

6. Configuración Crítica

Para que el sistema funcione bien, hay que ajustar dos parámetros:

- **Cascada (Cascade):** Define si al guardar/borrar un objeto, la acción se propaga a sus hijos.
- **Profundidad de Activación (Activation Depth):** Define cuántos niveles de objetos relacionados se cargan en la RAM al recuperar un dato. Evita cargar la base de datos entera o recibir `NullPointerException`s inesperados.

7. Colecciones y Relaciones

- **Adiós a las Tablas Intermedias:** Las relaciones N:M se resuelven con dos listas cruzadas (`List<Alumno>` en Curso y `List<Curso>` en Alumno).
- **Soporte Nativo:** `ArrayList` (ordenados), `HashSet` (sin duplicados) y `HashMap` (clave-valor) se guardan directamente.